

Concurrency for Modern CPUs

Lock-Free or Lock-based?

FEDOR G PIKUS



20
26





Modern Concurrency: Lock-Free Or not?

Fedor G Pikus

Coding Examples

- Benchmarks and concurrent data structures:
<https://github.com/fpikus/ConcurrentCpp>
- Benchmarks use Google Benchmark
<https://github.com/google/benchmark>





Why Lock-Free? Why Now?

CppCon 2017

2017



C++ atomics: from basic to advanced. What do they do?

Fedor G Pikus
Chief Scientist
Design2Silicon Division

September 25, 2017

really

Mentor
A Siemens Business

- Write your most demanding concurrent code lock-free
- Most demanding means highest contention (mostly)

2026

10 years



- Almost 10 years ago
- The machinery is all the same
- The basics are all the same
- The conclusions appear to be almost 100% wrong 10 years later
- What changed?
- And what do we do about it?



CppNow 2026

May 2026



- Locks are (mostly) fine
- Write low-contention code lock-free
- Hybrid synchronization for highest performance
 - Beware of latency...

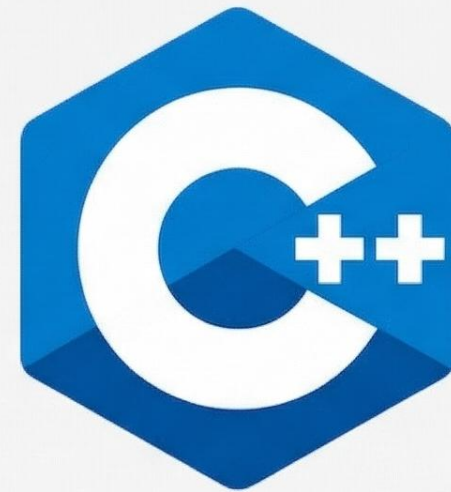
Sept 2026

10 years



- More data
- ~~Better explanations~~
- Better analysis
- How do we write high-performance concurrent code now?





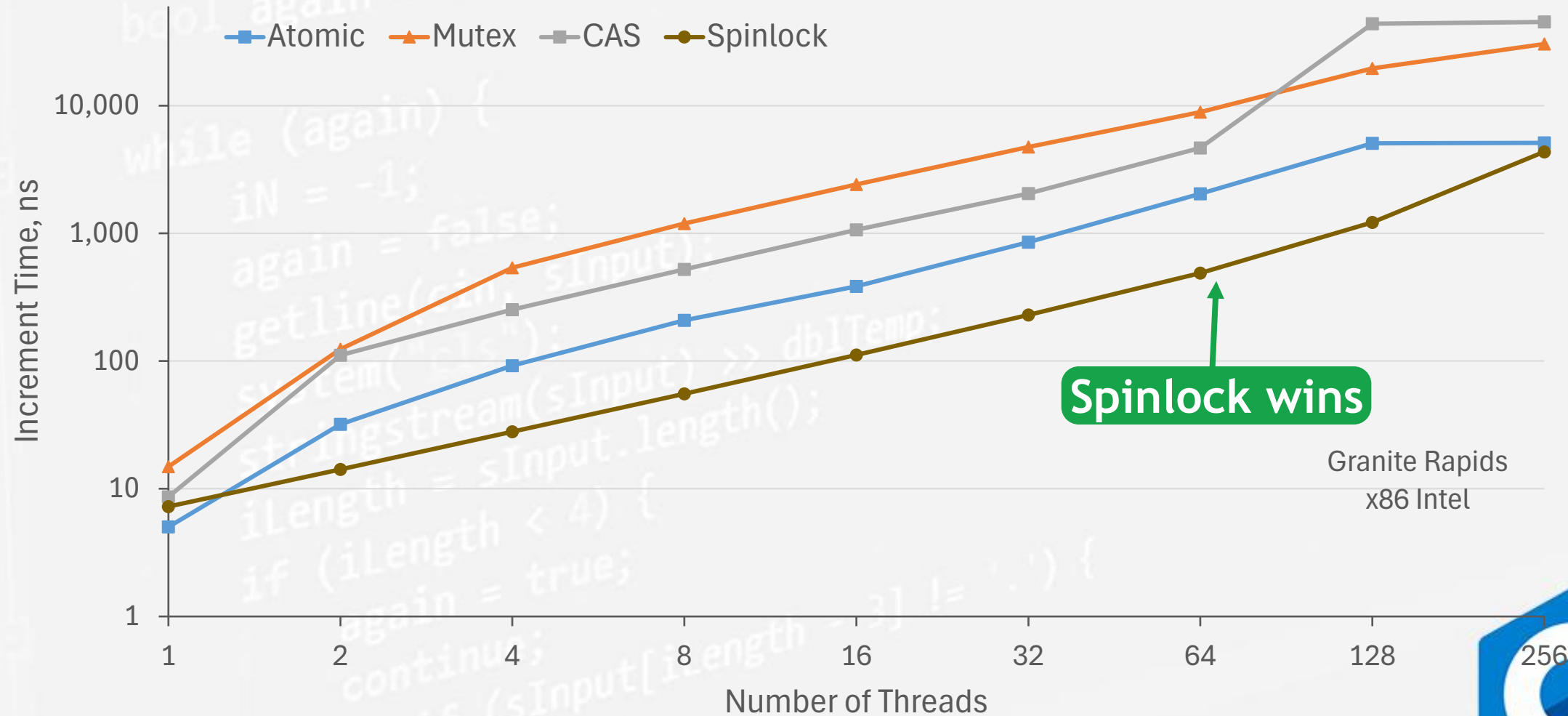
Is Lock-Free Faster?

Is Lock-Free Always Faster?

- In a wait-free program, all threads are making progress
 - At all times - that's as fast as it can be
- In a lock-free program, at least one thread is making progress
 - Possibly more than one, but usually not all - slower
- In a lock-based program, one thread is making progress
 - Even that is not guaranteed - the slowest
- Benchmark: increment a shared integer on all threads
 - Comparing wait-free vs lock-free is usually artificial
- Maximum contention - the hardest stress test



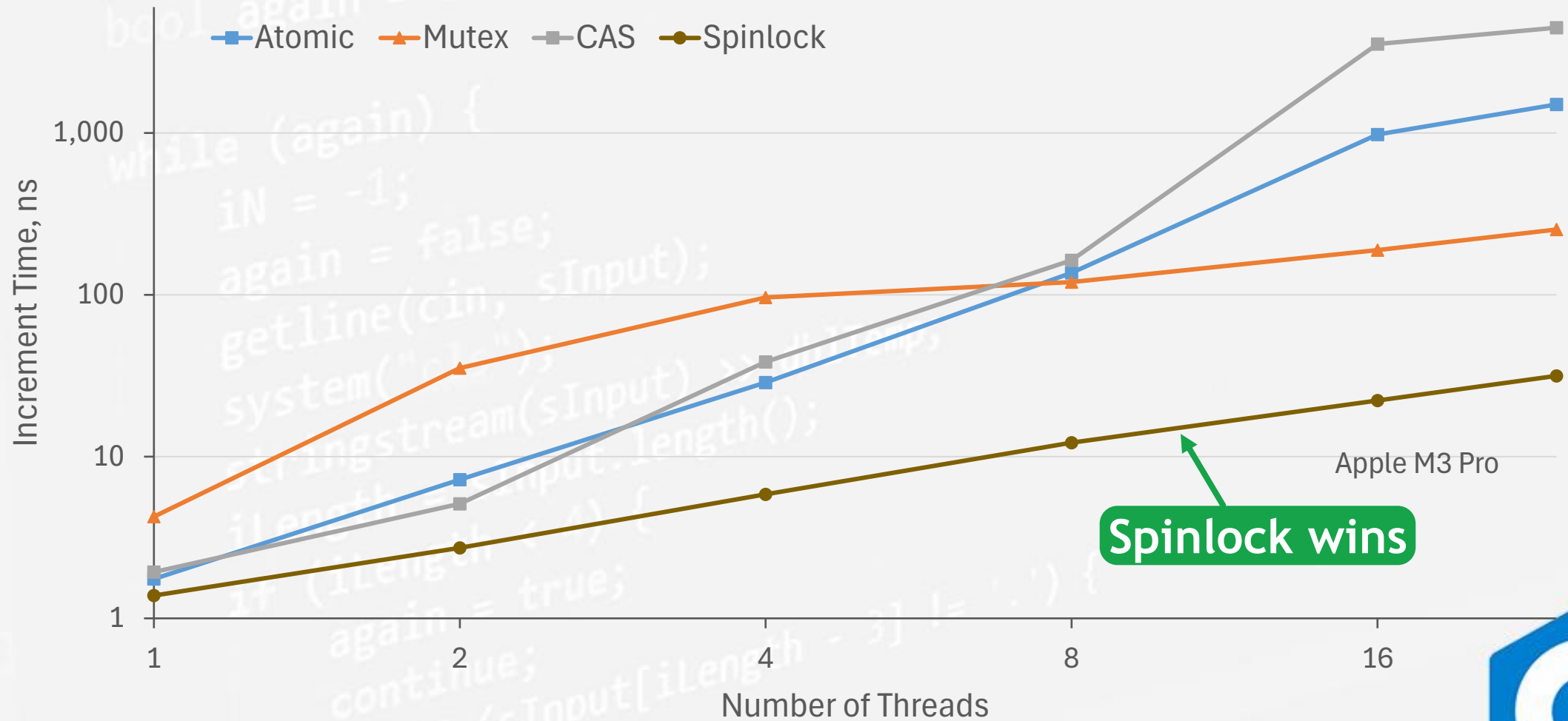
Is it?



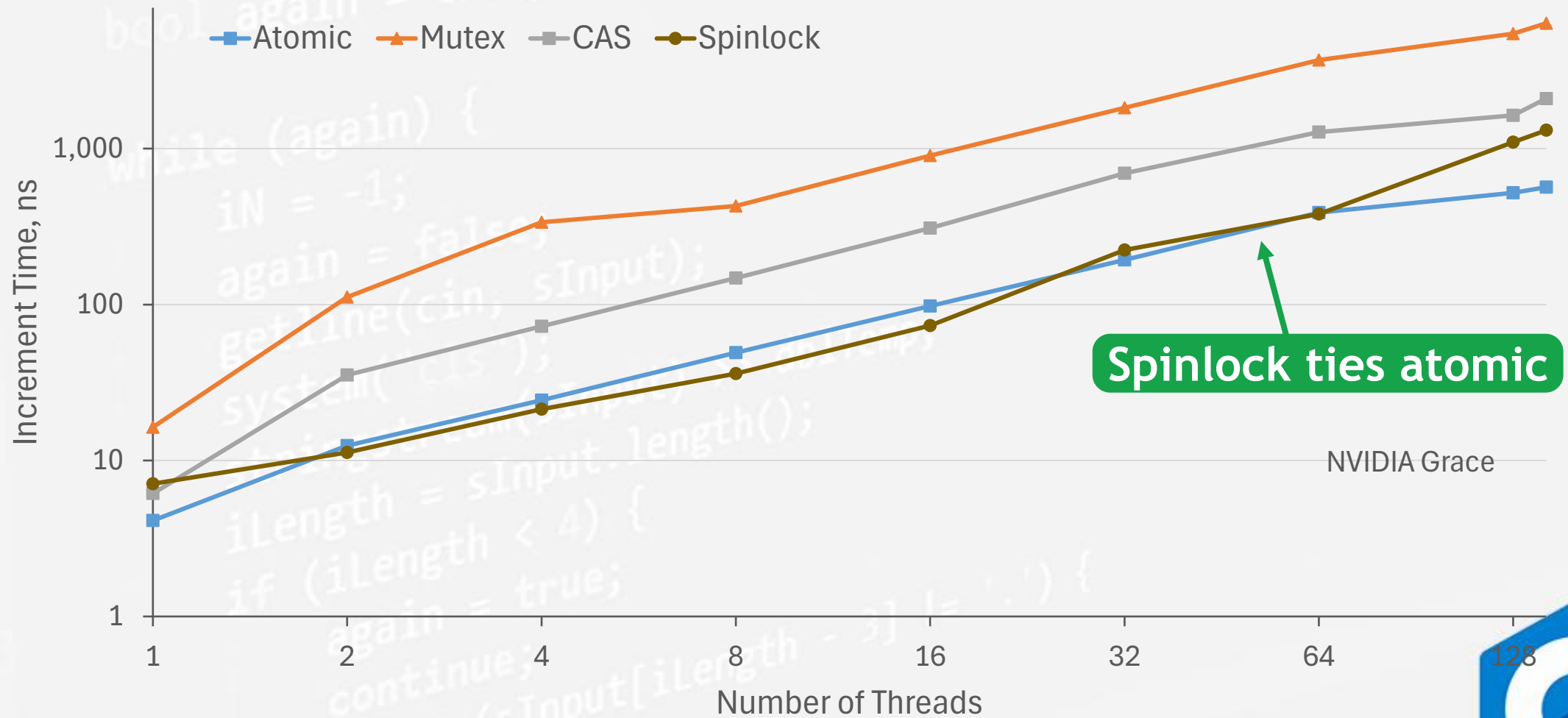
Granite Rapids
x86 Intel



Is it Hardware-Dependent?



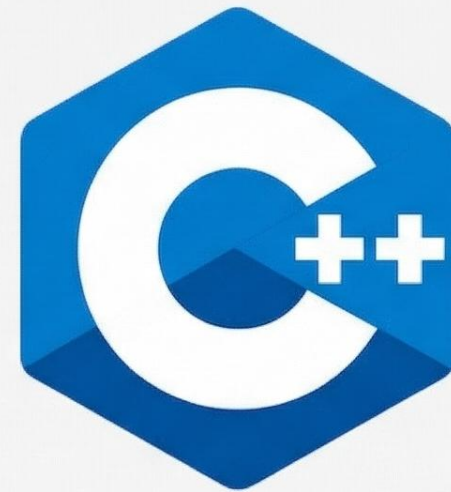
Is it ARM vs x86?



Is Lock-Free Faster?

- Under high contention, spinlock usually outperforms atomics
- Lock-free (CAS) is slower than wait-free
 - But very few computations can be wait-free
- Some locks are faster than others
- What makes a good spinlock?





What is the best lock?

How to Write Good Spinlock

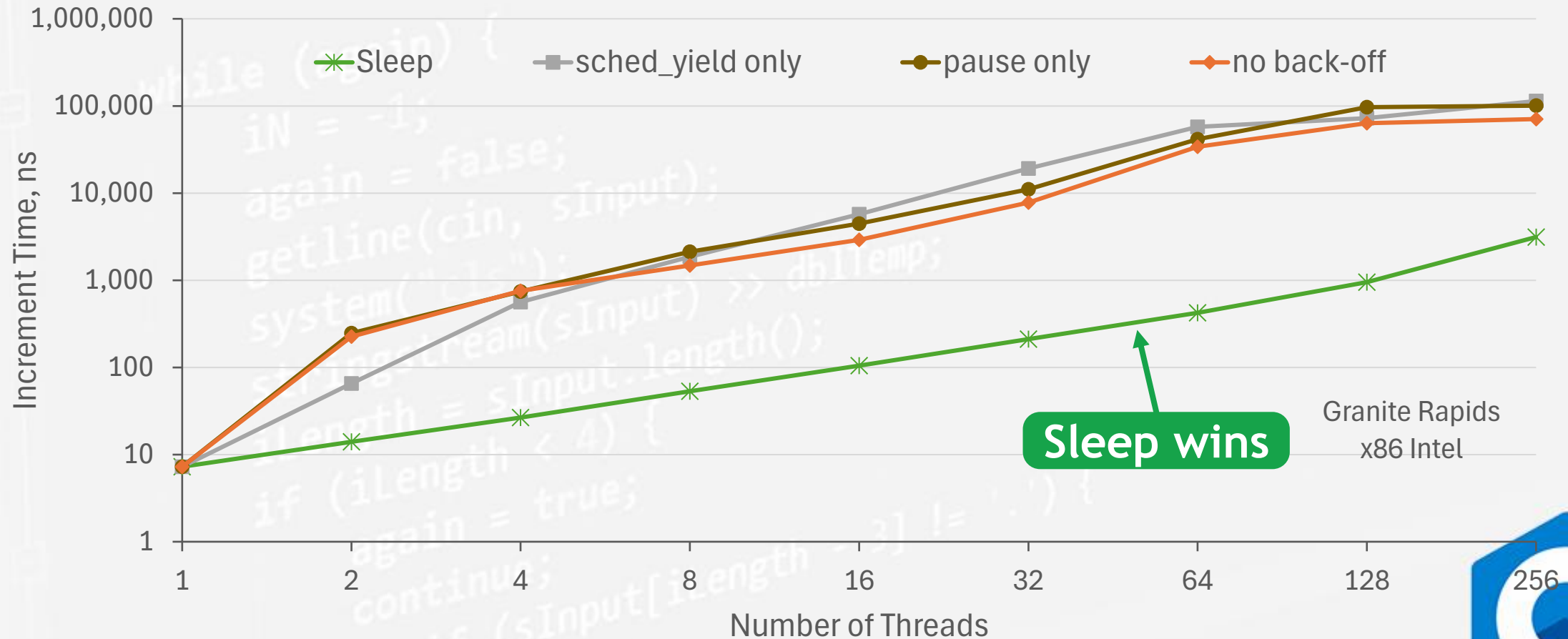
```
class Spinlock {  
    std::atomic<int> flag_ {}; // 1 - locked  
    void lock() {  
        while (flag_.load() || flag_.exchange(1))  
            backoff();  
    }  
    void unlock() { flag_.store(0); }  
};
```

Diagram annotations:

- No cache lock** (blue box) points to `flag_.load()`
- Lock** (red box) points to `flag_.exchange(1)`
- Yield scheduler** (orange box) points to `backoff();`
- Unlock** (dark blue box) points to `flag_.store(0)`



Back-off at high contention



Granite Rapids
x86 Intel

Cache Coherency - MESI



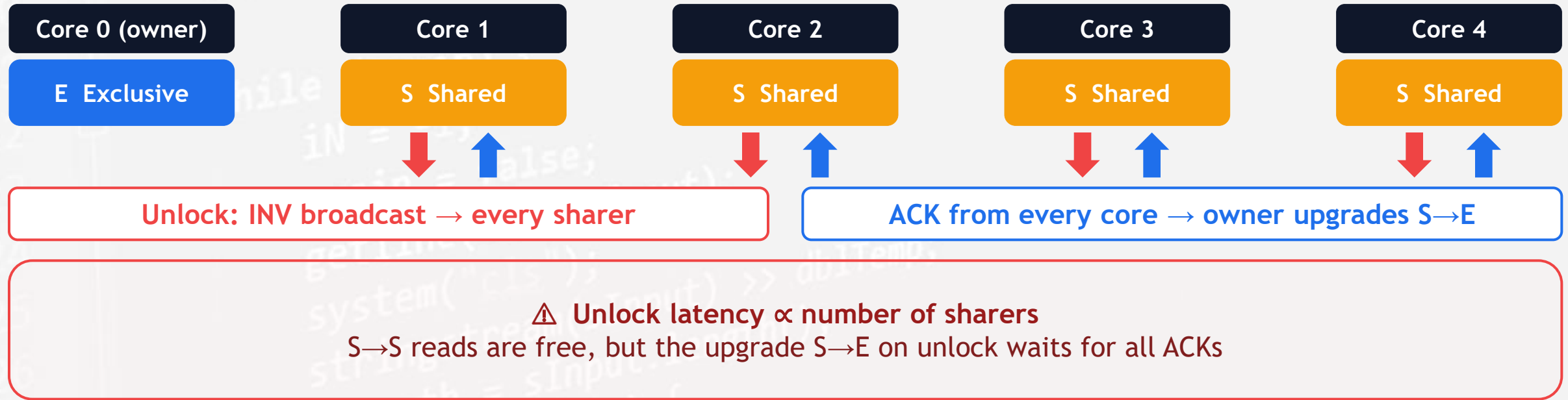
⚠ The write blocks until every ACK arrives

This is why writes feel slow: each invalidate is a round-trip across the interconnect, scaling with core count.



Spinlock with Read

Spinlock with pause — readers share, unlock broadcasts: read-spin holds S state, but unlock must invalidate all sharers

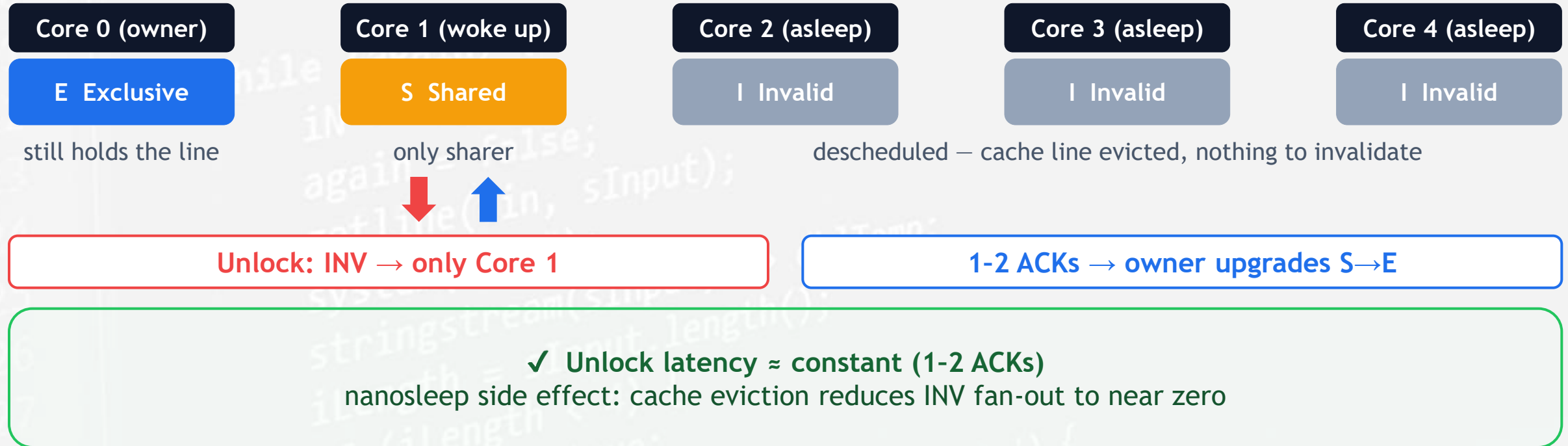


- ACK confirmations must be received from all cores!
 - Read with retries waits for RFO acknowledgements - don't give up too fast!
- To release the lock, core needs E state again (hidden cost of locks)



Spinlock with sleep

Spinlock with nanosleep — descheduled cores lose cache: context switch evicts lines → fewer sharers → fast unlock



At high contention, reducing coherence traffic dominates all other optimizations
Wait-free code still waits — it just waits in hardware, on the interconnect



Perils of Lock Release

- Thread acquires the lock: cache line is Modified on Exclusive
- Thread polls the lock (read): cache line transitions to Shared
- The hidden cost: **releasing** the lock
- When lock owner wants to write 0 to release the lock...
 - Cache is in M or E state: just write it
 - Cache is in S state:
 - Send Invalidate broadcast to all cores and wait for acknowledgement
 - Upgrade from S to E state and write (state changes to M)
- Paused threads keep their cache lines
- Sleeping threads don't issue reads (and may evict cache lines)



High Contention Regime

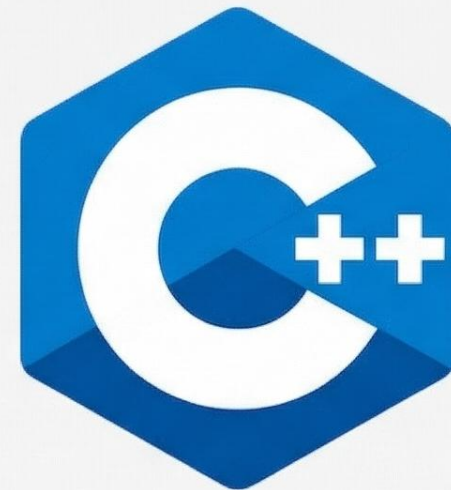
- Dominated by cache coherence protocol (MESI)
- Rapid backoff (sleep) is critical for spinlock performance
 - Without backoff (or with just pause) spinlock is worse than atomic
- To write into a cache line, the core must have that cache line in Exclusive state (or Modified state)
 - Writing changes state from Exclusive to Modified
 - Only one core can have Exclusive lock on the cache line
- Reading changes state to Shared
 - To transition back to Exclusive, invalidations are broadcast
 - Reader must acknowledge (ACK) and transition to Invalid



Why Locks are Faster

- At high contention, cache coherency (MESI) overhead is critical
 - Worse on Apple M* - cache sharing has long latency
 - Better (not enough) on server CPUs with complex interconnect
- Rapid backoff is essential for lock performance
 - Yield scheduler after few failures to lock
 - Experimentally `nanosleep(1ns)` gives best performance
- Aggressive backoff: fewer active reads - fewer invalidations
 - Apple M* uses different protocol, trades power efficiency for latency - this is why atomics are worse on M3





Are locks really faster?

Are locks really faster?

- We just measured it!
- At the highest contention
- The most challenging case
- If locks are better here, they are better everywhere
- The first rule of performance...



Are locks always faster?

- Real programs access shared data “in service” of the real work
 - High-contention sharing happens but not 100% of the time
 - Best case for shared increment is serial execution (zero overhead)
- We can model it by mixing operations on local and shared data

```
unsigned long x;  
spinlock s;  
for (...) {  
    unsigned long xl = 0;  
    xl = SharedUpdate(x, xl);  
    xl = LocalWork(xl);  
}
```

Diagram annotations:

- An orange box labeled **Shared** has arrows pointing to the variable `x` and the `SharedUpdate` function.
- A green box labeled **Per thread** has an arrow pointing to the `LocalWork` function.



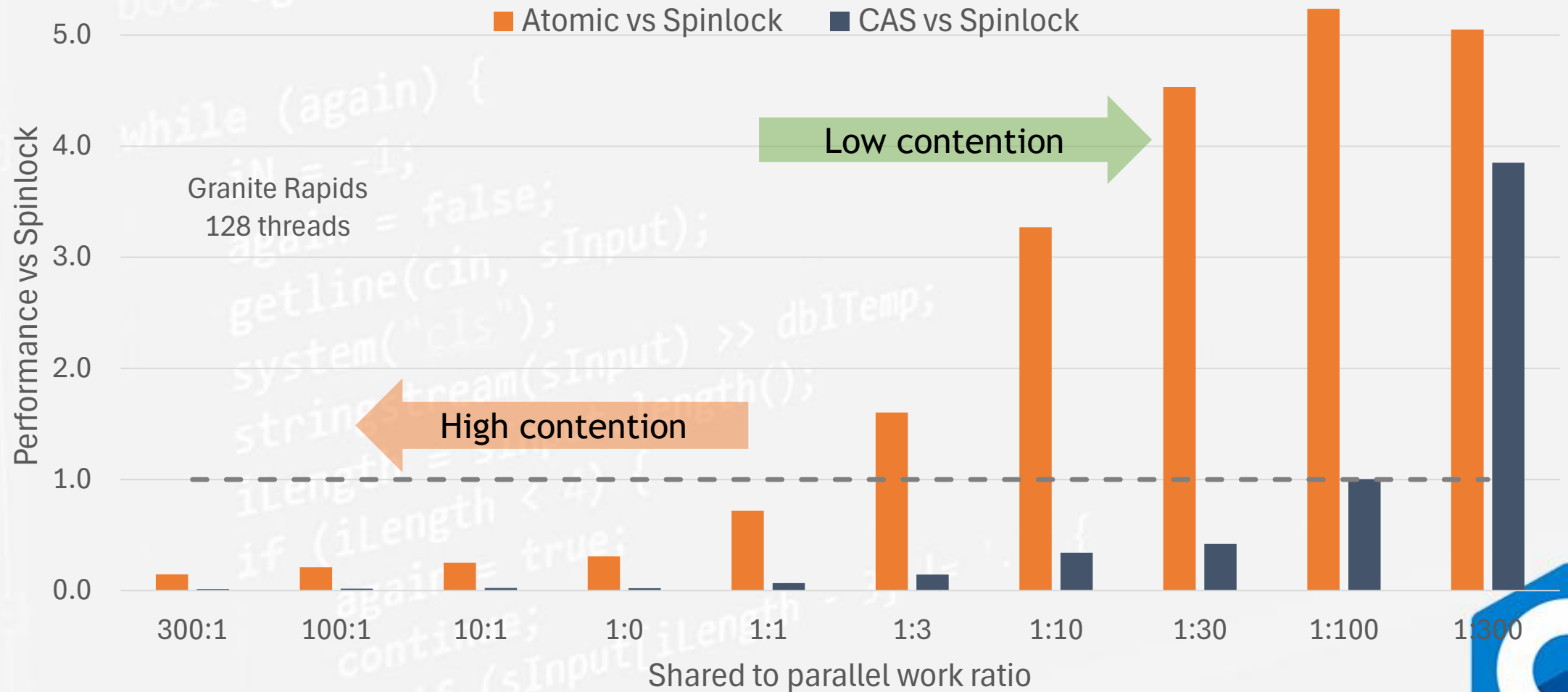
Benchmarking Synchronization Overhead

```
unsigned long n = 0; ← Per thread
for (...) {
    unsigned long dx = 1;
    for (size_t i = 0; i != nshared; ++i) {
        x.fetch_add(dx, std::memory_order_relaxed);
    }
    double y = x;
    for (size_t i = 0; i != nwork; ++i) {
        y = sin(++n + cos(y));
    }
    dx = y + 1;
}
```

- n is the measure of work on this thread
 - Depends on the ratio of **shared** and **local** computing
- Use lock or CAS loop instead of `fetch_add` to benchmark other synchronization options



Good, Better, Best?



Why?!

- At high contention, spinlock is the best
 - Better than the single atomic instruction (increment)
 - Aggressive backoff is the main reason (reduces invalidation traffic)
- At low contention, spinlock is the **slowest**
 - Throughput degradation out of proportion to the shared portion
 - 10:1 parallel/shared ratio → 3x performance degradation vs atomic
 - Spinlocks aren't just slow - they degrade performance of surrounding code which has no shared data
- Spinlocks degrade performance of surrounding code
- How? Why?



Profiling Low-Contention Regime

- On x86 three mechanisms are suspect:
 - Store Buffer stalls (r40a6)
 - Machine Clears - memory ordering (r02c3)
 - Branch misses

	SB Stalls*	Machine Clears*
Spinlock	2731	35.4
Atomic	0.1	0.6
CAS	0.3	5.1

(*) per 1M instructions



Spinlocks vs code

- Where are store buffer stalls?

Overhead	Command	Shared Object	Symbol
49.75%	overhead_bm	libm-2.28.so	[.] 0x0000000000007e6b4
24.78%	overhead_bm	libm-2.28.so	[.] 0x0000000000007e6b8
8.03%	overhead_bm	overhead_bm	[.] sin@plt
3.74%	overhead_bm	libm-2.28.so	[.] 0x0000000000007eee3
2.11%	overhead_bm	libm-2.28.so	[.] 0x0000000000007eee9
		...	
0.49%	overhead_bm	libm-2.28.so	[.] 0x0000000000007e846

- Spinlock “poisons” thread-local code
 - Unlock (write) blocks store buffer for a long time
 - Parallel computations wait on SB and stall

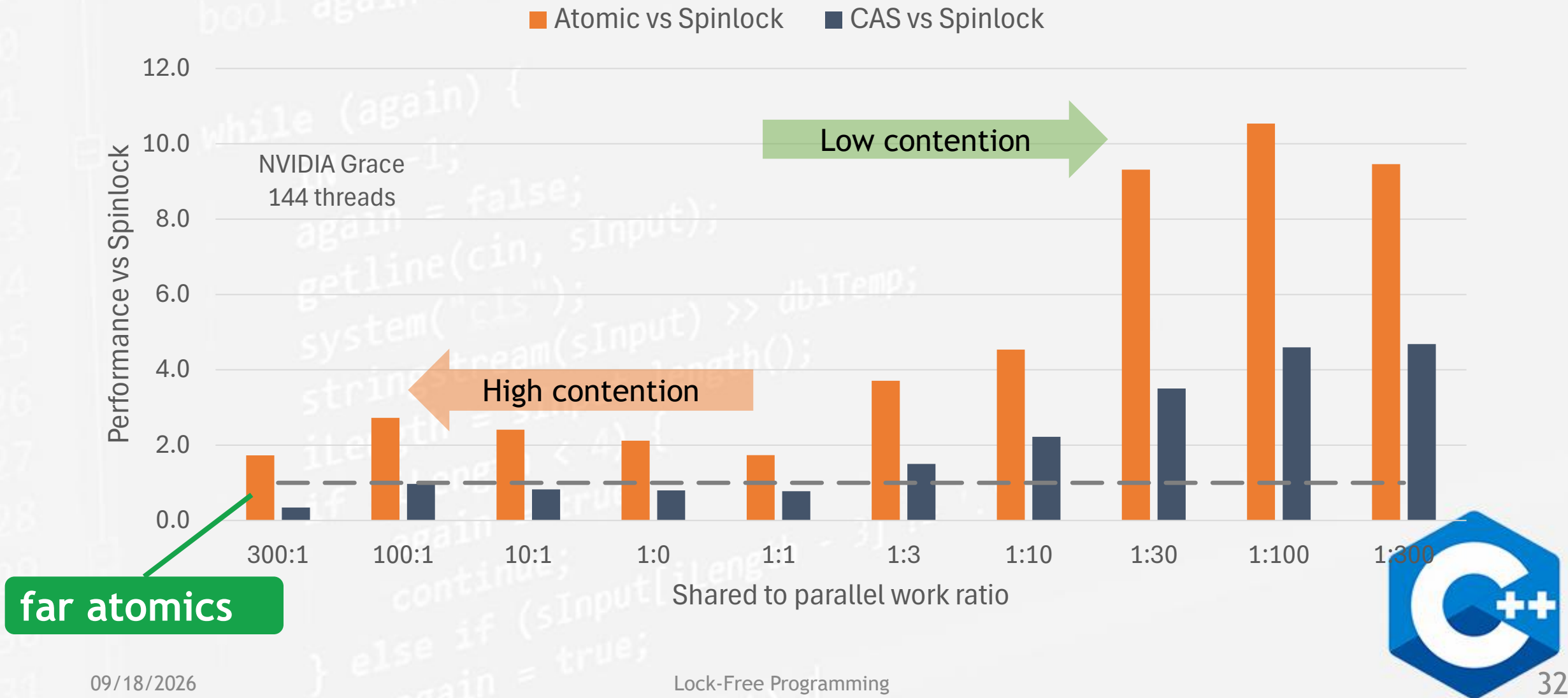


Perils of Implicit Synchronization

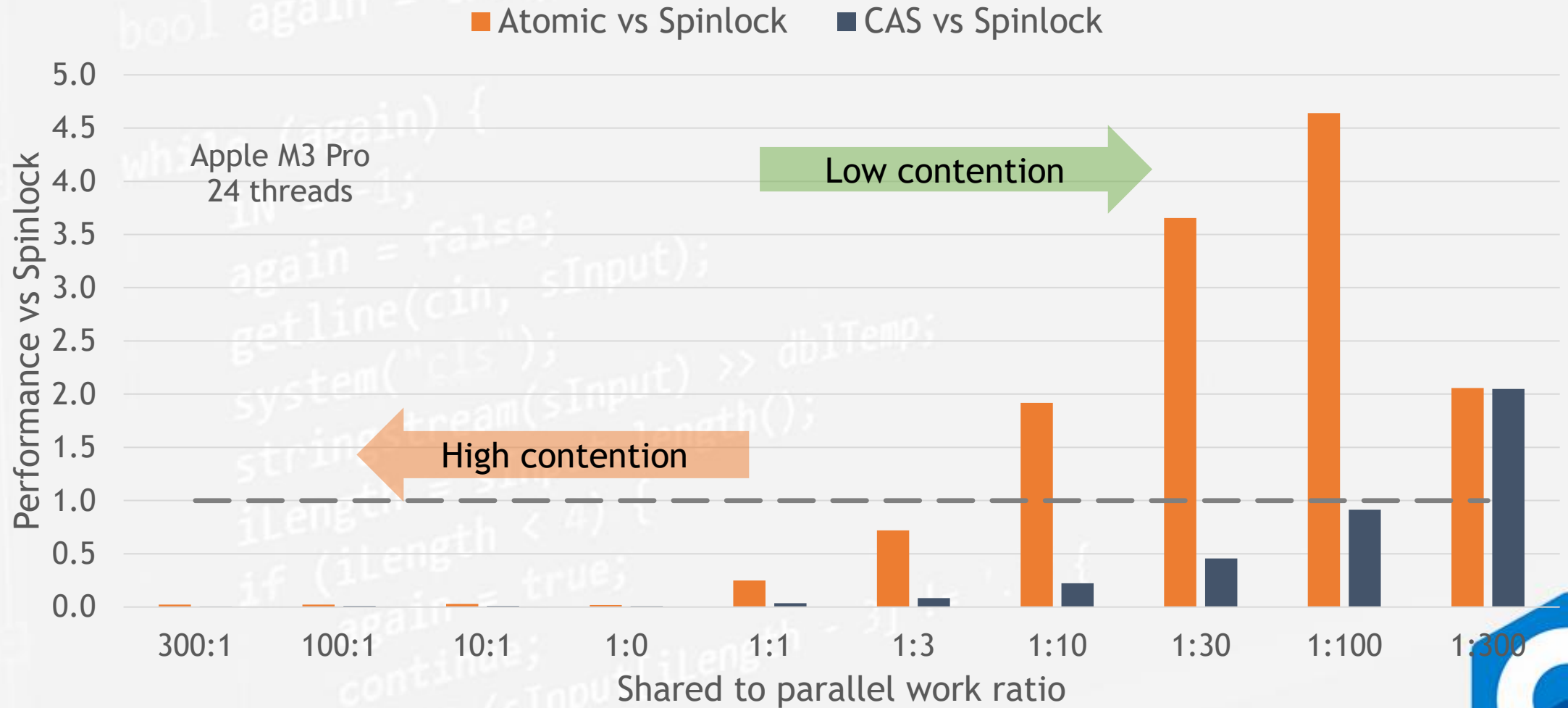
- Locks maintain exclusion by establishing happens-before and happens-after relations between independent variables
 - Lock flag and protected data
- Locks need memory fences to maintain critical section
- On x86, exchange is a full memory fence - Load-Store Unit has to halt and wait for Store Buffer to drain
- CAS and atomics operate on a single shared variable
 - Fuses control and data into a single instruction
- Implicit Execution Dependency is bad for concurrency
- Spinlock code is limited by store buffers not retirement
 - Pipeline stalls before ROB fills up - fewer ROB stalls
 - Less computing is done



ARM has out-of-order stores but...



ARM has out-of-order stores but...

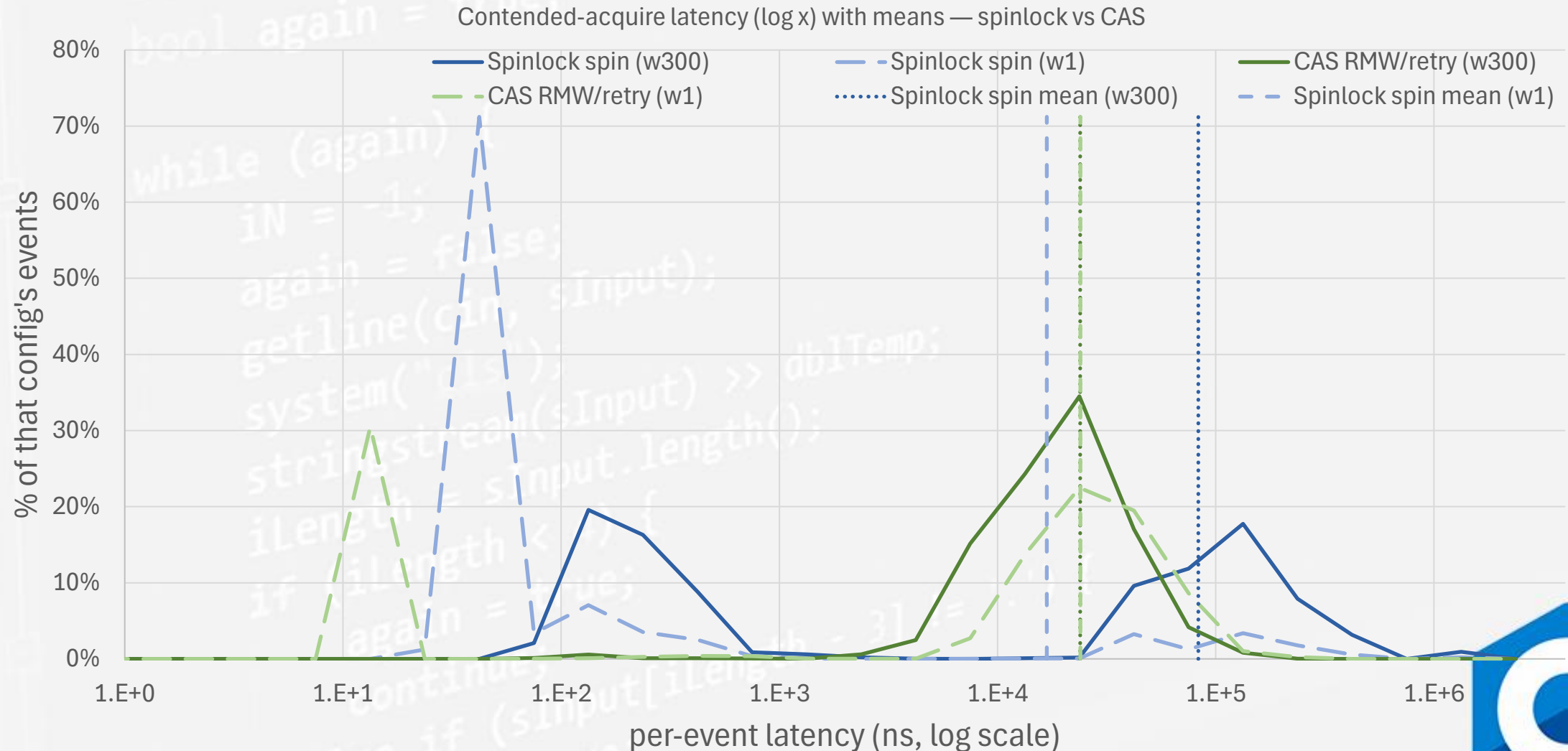


Key Takeaway about Locks

- 1 At high contention, a good spinlock beats CAS**
Usually beats atomics – though fast atomics exist on some hardware
- 2 At moderate contention, CAS pulls ahead**
A single atomic RMW is even faster, until results converge at very low contention
- 3 The crossover holds across x86 and ARM**
Intel, AMD, Neoverse V2, M3 – the point moves, but a crossover always exists
- 4 The result is universal – the explanation is not**
No single mechanism explains this behavior on all hardware



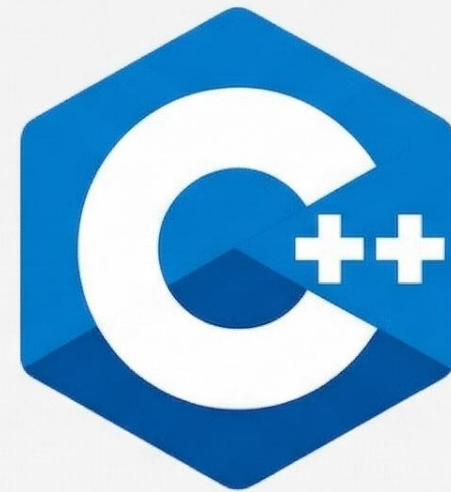
What does machine do all this time?



Spinlock “poisoning”

- Order-dependent (x86-specific): spinlock stores stall SB
- Resource starvation (general):
 - Long-latency instructions occupy limited Load-Store Unit (LSU) slots
 - ROB capacity is limited and long-latency stores stall it
- Memory clears (measured on x86): spinlock read speculatively executed then invalidated by release-store
 - Test-and-Test-and-Set (TTAS) spinlock is necessary to avoid RMW storm, but it creates a speculation hazard on the read side
- Not enough to explain all measurements in all regimes
 - Results are clear but the mechanism is not...





What is the best lock?

What is the best spinlock?

- High contention: sleep back-off
 - Slight machine-dependent differences for different sleep stages
 - Does not depend on the workload (write, read, RMW - all the same)
- Very low contention: mostly doesn't matter
- Low/moderate contention: this is where things get messy
 - Yield back-off generally outperforms sleep back-offs
 - Write-heavy workloads: ~30% differences on most hardware
 - Read-only workloads: 3x differences are common
 - Different hardware produces different results
 - Best lock depends on the workload and contention



What is the best spinlock?

- Sleep back-off is the “safe” choice
 - If you’re wrong about “low” contention, everything else dies
 - When it’s not optimal, it’s usually “not too bad”
- For low contention, may optimize for readers vs writers:
 - This is not a read-write lock!
 - Same flag but different back-off policies
 - Readers use yield, writers use sleep



Optimizing critical section

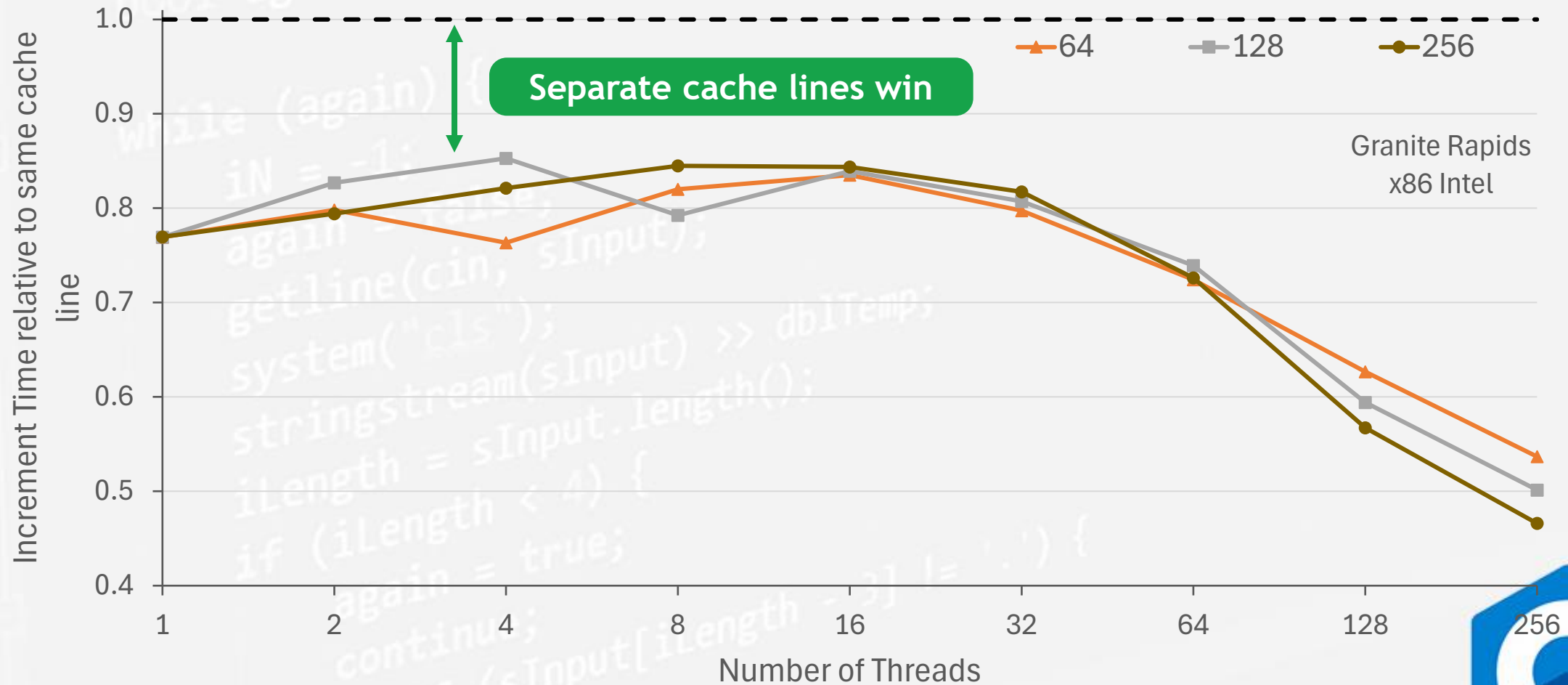
- We forgot something...
 - Spinlock - ✓, parallel workload - ✓, shared workload - ?

```
unsigned long x;  
spinlock s;           // Guards x  
...  
lock_guard g(s);  
++x;
```

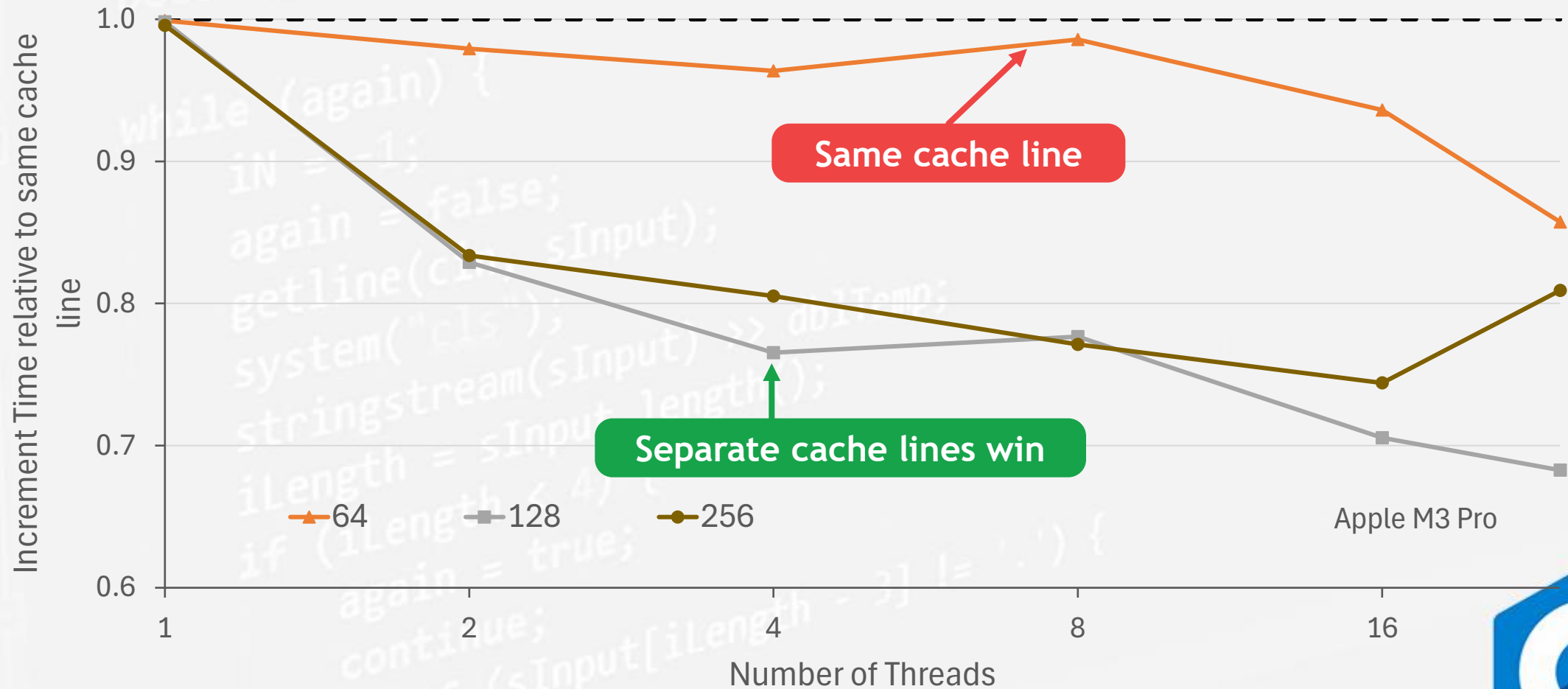
- Do spinlock and shared data go on the same cache line?
- Duh - of course, the core that has the lock already fetched that cache line, anything else is a cache miss



Measure, Don't Guess



Measure, Don't Guess

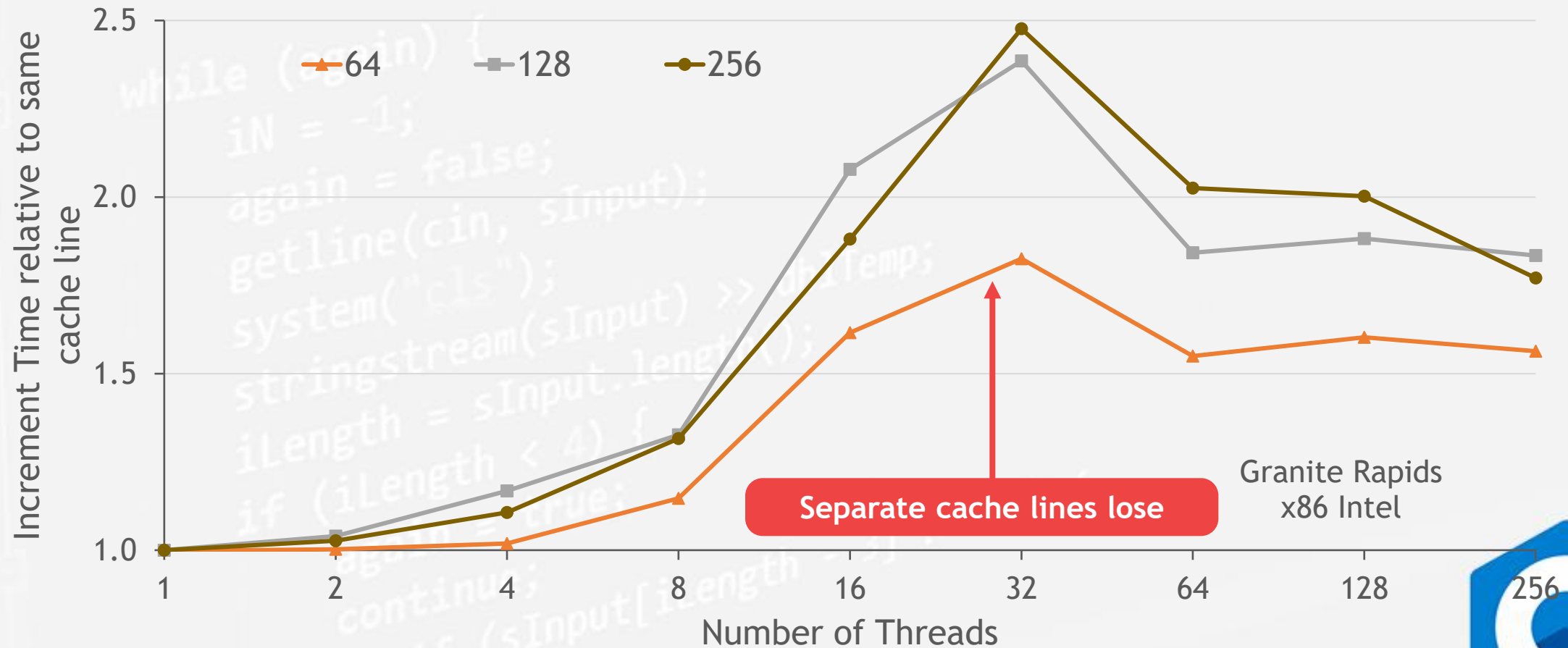


Optimizing critical section

- At high contention, nobody owns spinlock cache line for long
- Other cores read the lock - cache line becomes Shared (S)
- Write access requires Exclusive ownership (E)
 - Other cores constantly pull away data cache line if it's shared
- This is true only for writer workloads!
- This is true only at high contention when concurrent lock attempts are frequent
 - At low contention, it's a competition between a guaranteed cache miss (different lines) and an unlikely second RFO (same line)



Critical Section - Low Contention



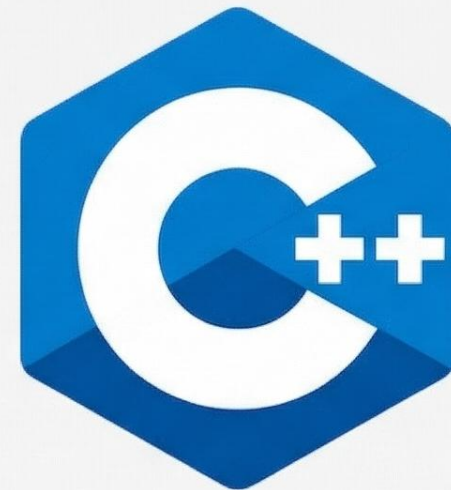
Granite Rapids
x86 Intel



Optimizing critical section

- At high contention, spinlock and payload should be separate
 - No core owns spinlock cache line for long
 - Other cores read the lock - cache line becomes Shared (S)
 - Payload on separate cache line - no competition for that line
- At low contention, spinlock and payload share cache line
 - Lock owner (usually) owns the cache line
 - Payload on the same cache line avoids a cache miss





Using locks efficiently

Not everything is lock's fault

- Example - concurrent maximum

Lock-free (not wait-free)

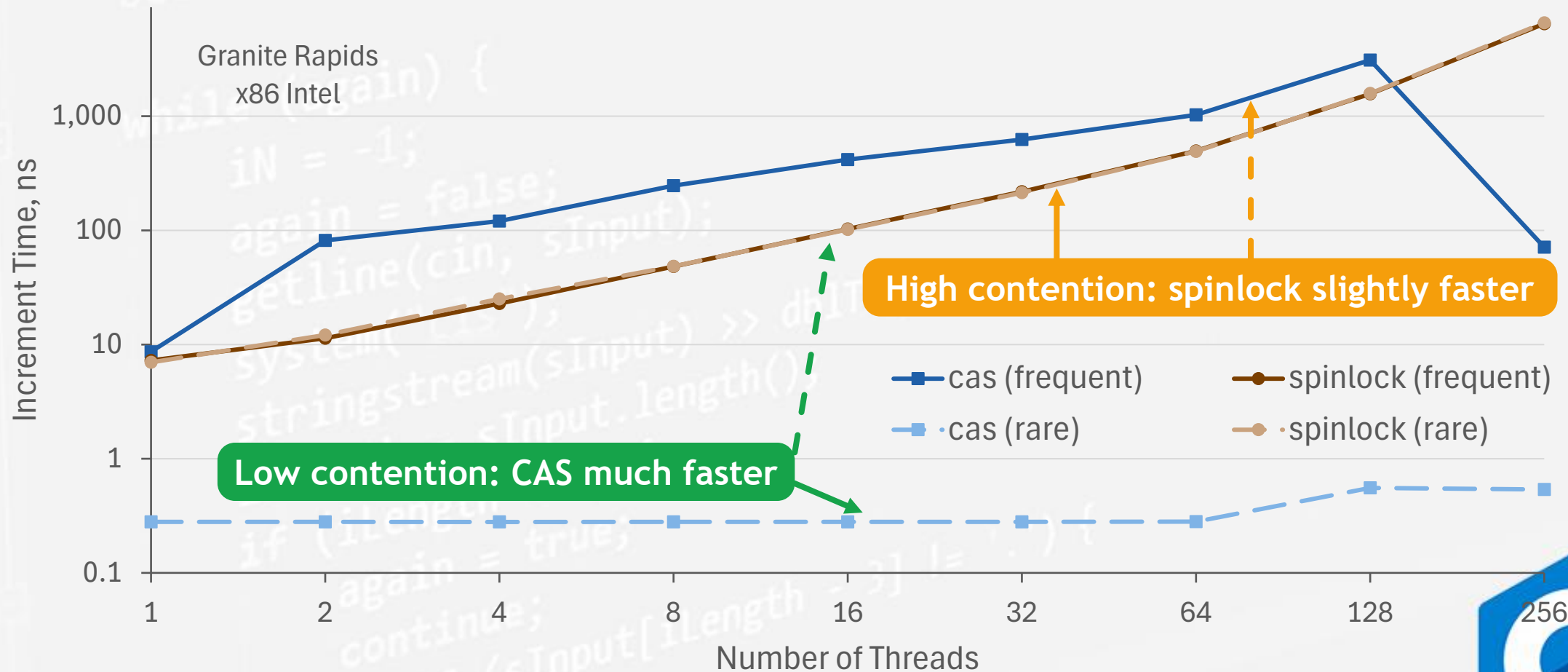
```
std::atomic<long> nmax;  
  
long n = ... new value ...  
long cur = nmax.load();  
while (  
    n > cur &&  
    !nmax.compare_exchange(cur, n)  
) {}
```

Lock-based

```
long nmax {};  
Spinlock S;  
long n = ... new value ...;  
  
{  
    std::lock_guard L(S);  
    if (n > nmax) nmax = n;  
}
```



Is Lock-Free Faster?



Inefficient code is inefficient

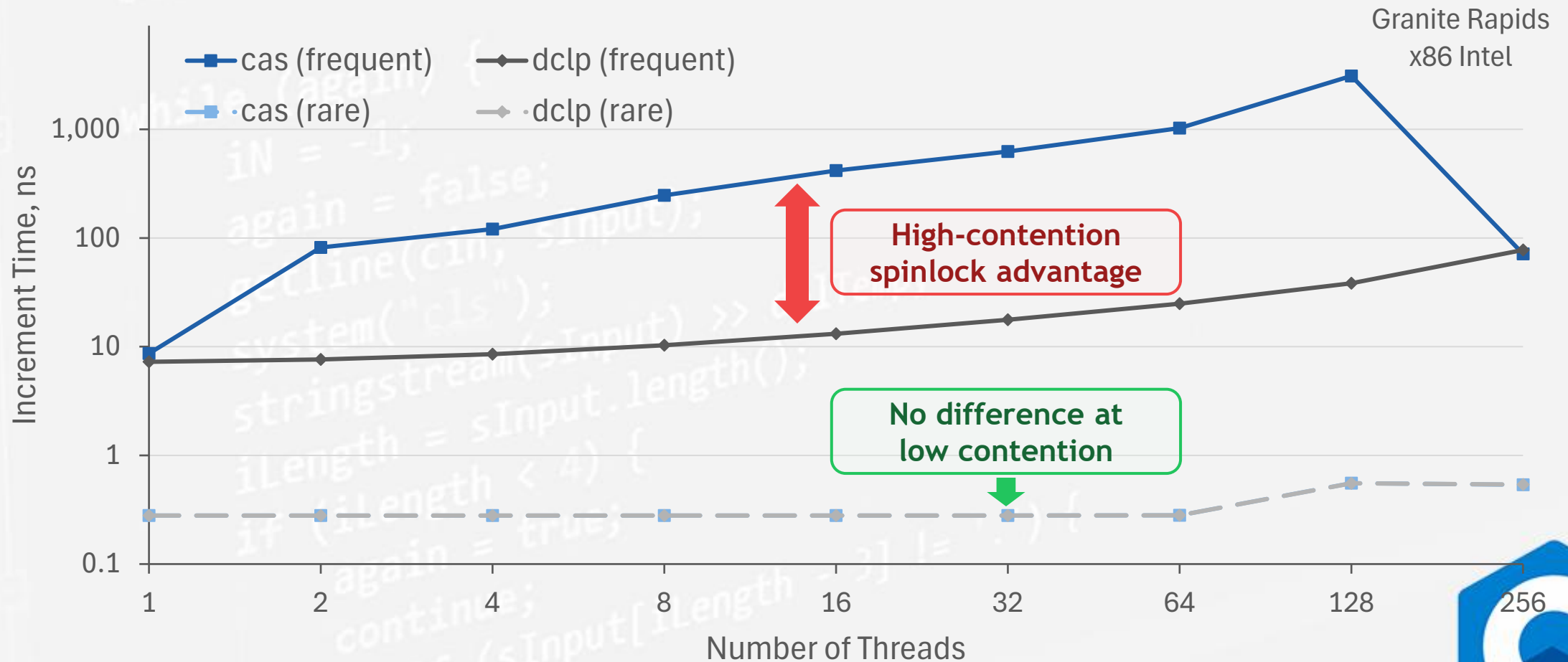
- The problem isn't the locks...

```
std::atomic<unsigned long> nmax {};  
Spinlock S; // Guards nmax  
// ... on all threads ...  
unsigned long n = ... new value ...  
if (n > nmax.load(std::memory_order_relaxed)) {  
    std::lock_guard L(S);  
    if (n > nmax.load(std::memory_order_relaxed))  
        nmax.store(n, std::memory_order_relaxed);  
}
```

- Double-Checked Locking Pattern (DCLP)



Well-written locks are fast



Lock-Free Style, with Locks

- The problem isn't always the locks
- Traditional lock-based code locks too much
 - Concurrent queue: pop head under lock
- You can write CAS-style code with locks
 - Concurrent queue: advance head under lock, copy data in parallel
- Often this results in DCLP (or similar) code:
 - Use atomics and read-only operations outside of lock
 - Critical section protects **modification** of the atomic variable
 - And nothing else



Main Takeaway

- Locks work better at high contention
 - backoff is essential for high-contention writes
 - Read with retries is critical - need to wait for ACKs on RFO
 - Hidden cost of the lock is its release
- DCLP (“lock-free style”) techniques minimize critical section
- Lock-free works better at low contention
 - Cost of the lock doesn’t matter - disruption of execution flow matters
 - Implicit synchronization: lock variable and guarded variable
 - Complex order restrictions disrupt CPU pipeline
- Locks rule average latency - lock-free rules tail latency



Guidelines

- Use locks for high-contention sharing
 - Except when bounded latency is critical
- Use locks for complex synchronization at low contention
 - One CAS is faster than locks
 - Two CAS on separate variables are usually slower
- Optimize lock-payload cache line sharing
- Use simple lock-free synchronization at low-contention



Designing a Concurrent Data Structure

High-contention domain

Locks for complex synchronization

Spinlock
acquire / release

Shared payload
complex updates

`std::atomic<T>`
written under lock

Lock + payload + handoff atomic share one cache line: one transfer per critical section

Low-contention domain

Shared atomics, but hardly ever contended

`std::atomic<T>`
read-only here

`std::atomic<A>`
shared, rarely
raced

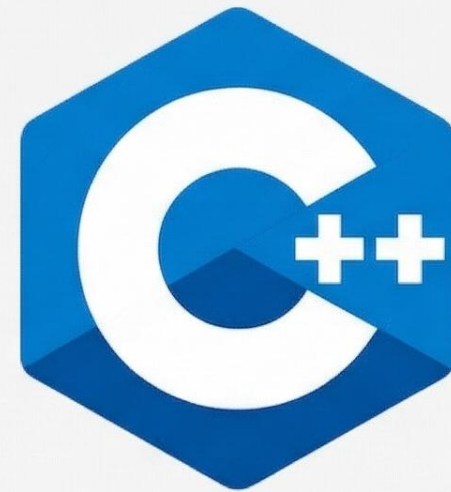
`std::atomic<B>`
shared, rarely
raced

(a) Shared, but two threads rarely try to modify the same one: a single CAS wins

(b) The handoff is read-only here: peek with an atomic load, no lock; take the lock only when the peek says it's worth it

⚠ **Handshake:** the handoff is atomic even though it is written under the lock — readers outside the lock can safely peek at it





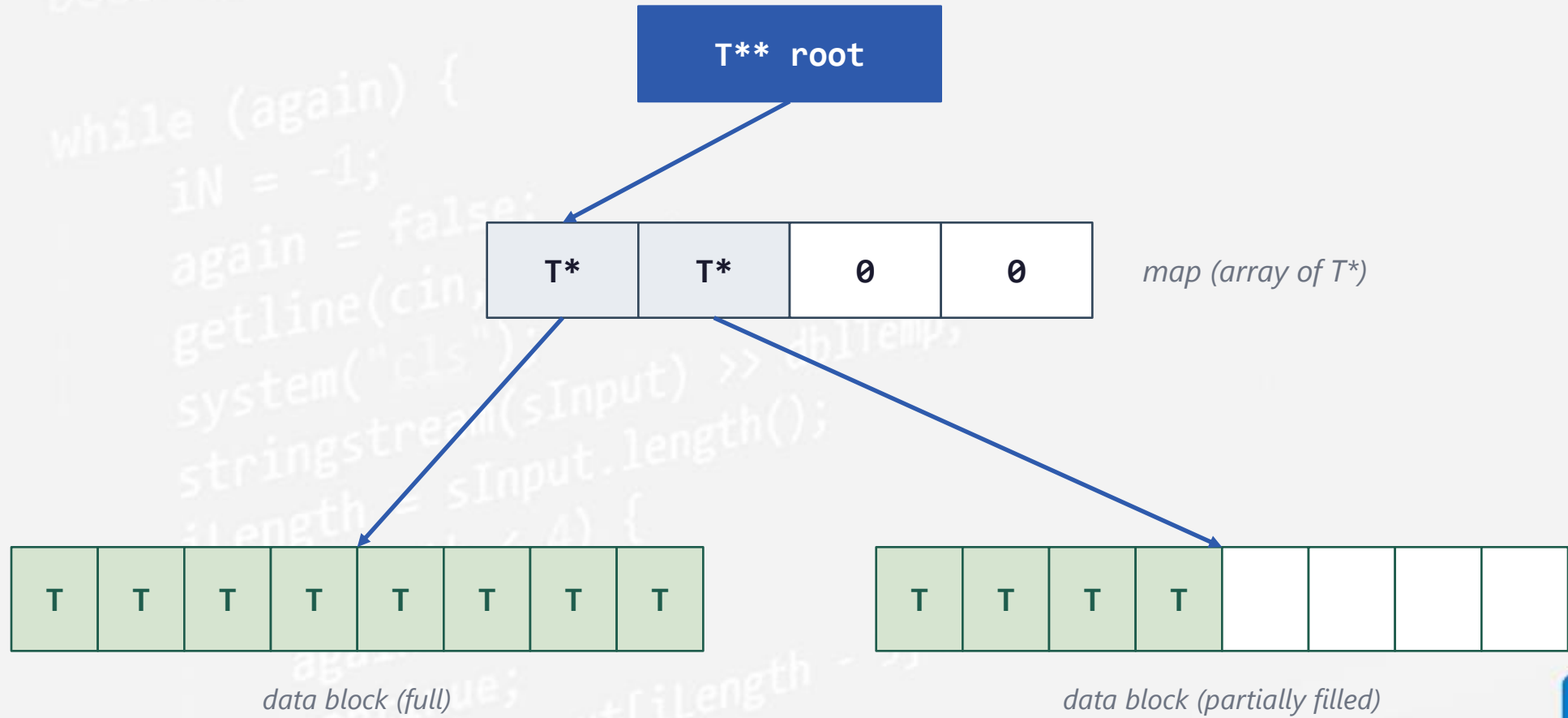
Concurrent Data Structures for Modern Hardware

Thread-safe growable “array”

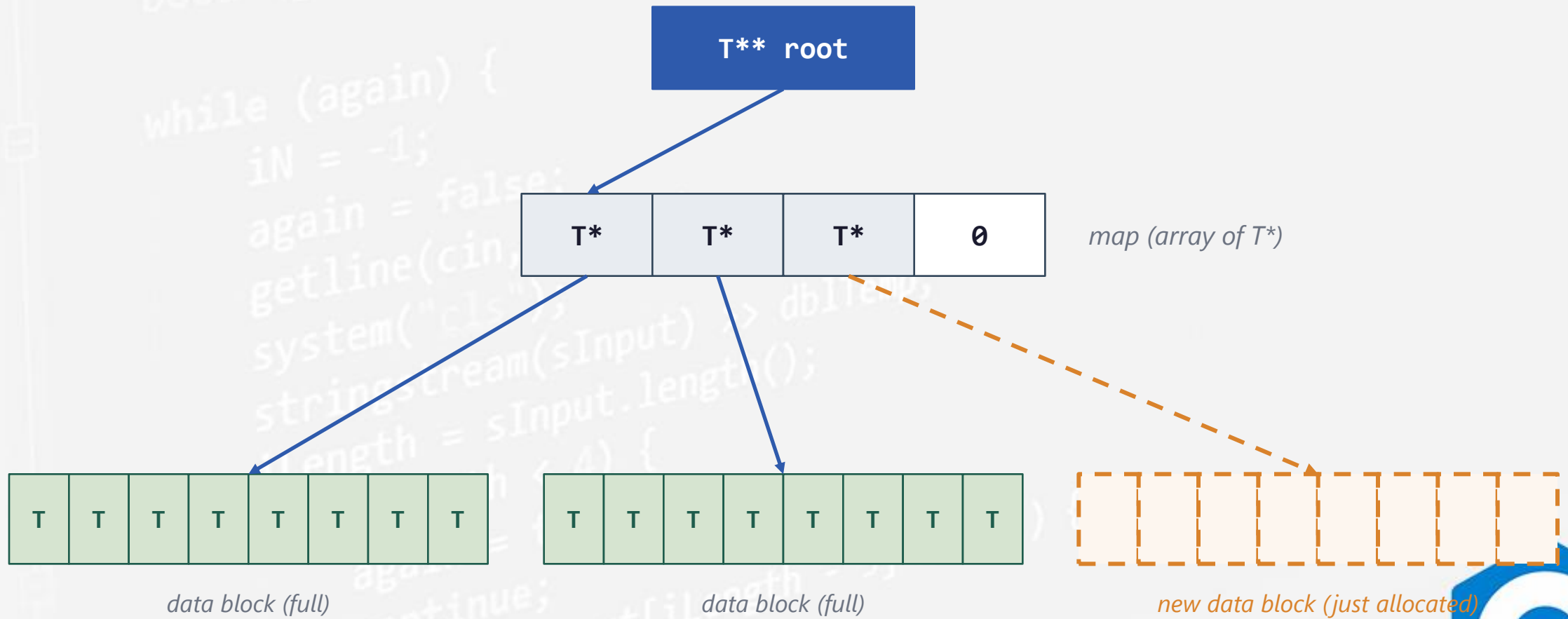
- Frequently needed: thread-safe vector-like container
 - Increasing size is thread-safe
 - Accessing elements is thread-safe
 - Concurrent access to the same element is client’s problem
- Not `std::vector` - reallocation moves data memory
- Deque? Growth does not invalidate existing elements
- Not `std::deque` - reallocation moves internal memory
 - Ideally, all calls to `operator[]` run concurrently and non-blocking
- Custom deque, then



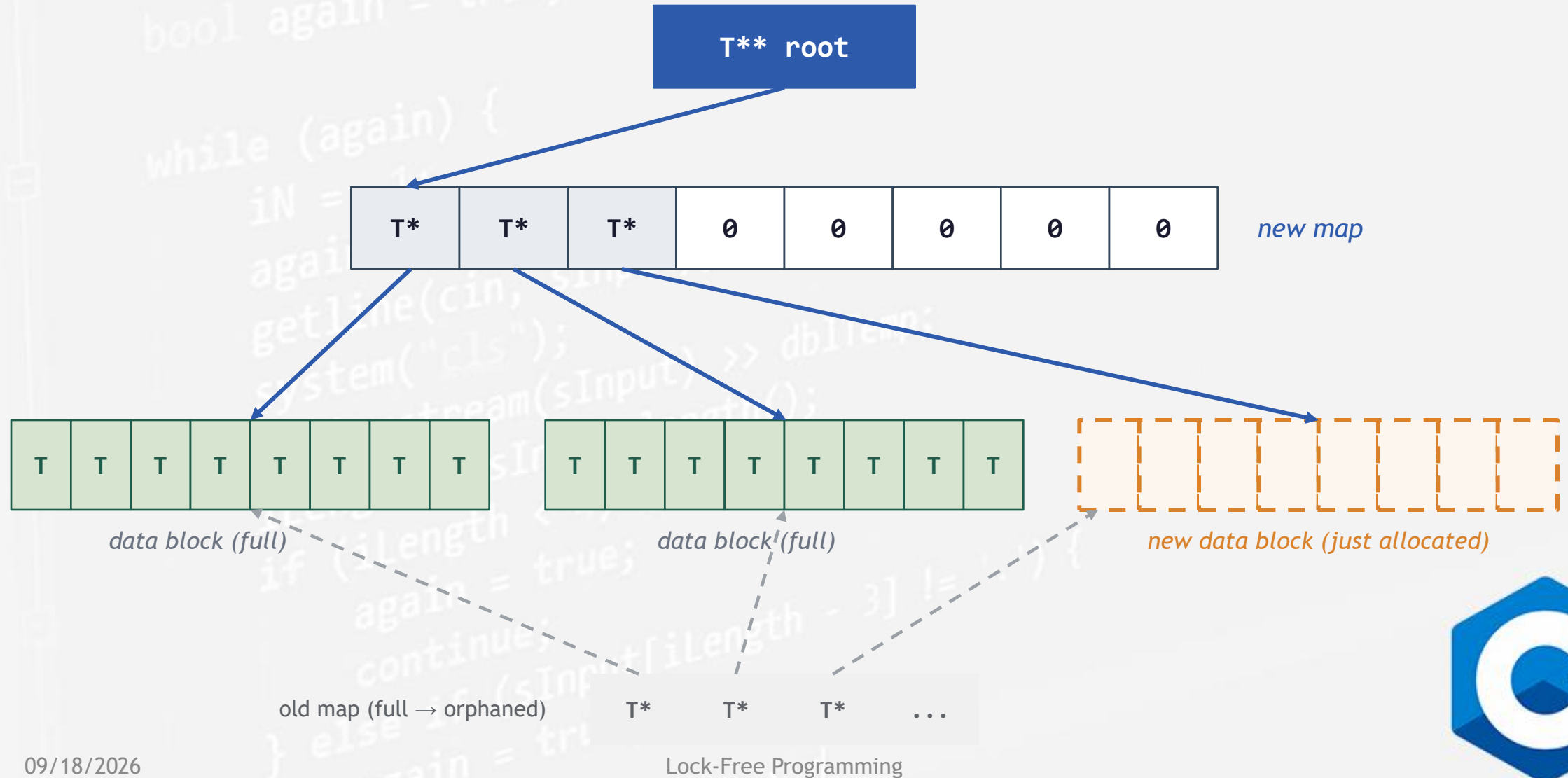
Deque structure



Growing a deque



Why not std::deque?



Concurrent Deque

- Data is block-allocated and never moves
- Atomic Root pointer: all accesses begin from the root
 - Root can change (atomically) to point to new map
- Nothing is ever deleted:
 - Resize to shrink not supported (other threads may access “old” data)
 - Old maps cannot be safely collected until destruction (minor issue)
- Map pointers are not atomic and never change once created



Concurrent Deque Use cases

- Each thread reserves some number N of elements
 - Resize up is thread-safe
 - Each thread uses its range of elements, or
 - Client arranges exclusive (write) access externally
- Any thread can check current size
 - This thread can access all elements in range
 - Client arranges exclusive (write) access externally
- Element access is low contention and wait-free (atomics)
- Resize is rare but hard to do without locks, so spinlock



Concurrent Deque - Interface

```
template <typename T> class Deque {  
    public:  
    Deque (...) {...} ← Standard stuff  
    void resize(size_t new_size); ← Locking: [0, new_size) visible  
    T& operator[](size_t i); ← Non-locking  
    const T& operator[](size_t i) const; ← Non-locking  
    size_t size() const; ← Non-locking: [0, size()) visible  
};
```



Concurrent Deque - Data

```
template <typename T> class Deque {  
    alignas(64) mutable spinlock s_;  
    alignas(64) std::atomic<size_t> size_ {};  
    alignas(64) std::atomic<T**> map_ {};  
};
```

Some const methods lock

Atomic handshake

Root pointer



Concurrent Deque - Fast Path

- Call `size()` and use elements known to exist - wait-free!

```
template <typename T>
size_t Deque<T>::size() const {
    return size_.load(std::memory_order_acquire);
}

template <typename T>
T& Deque<T>::operator[](size_t index) {
    T** m = map_.load(std::memory_order_acquire);
    size_t block_idx = index >> BlockShift;
    size_t local_offset = index & BlockMask;
    return m[block_idx][local_offset];
}
```



Concurrent Deque - Resize

```
template <typename T>
void Deque<T>::resize(size_t new_size) {
    size_t cur_size = size_.load(acquire);
    if (new_size <= cur_size) return;
    std::lock_guard lock(s_);
    cur_size = size_.load(relaxed);
    if (new_size > cur_size) { ... grow ... }
}
```

Slideware

DCLP!

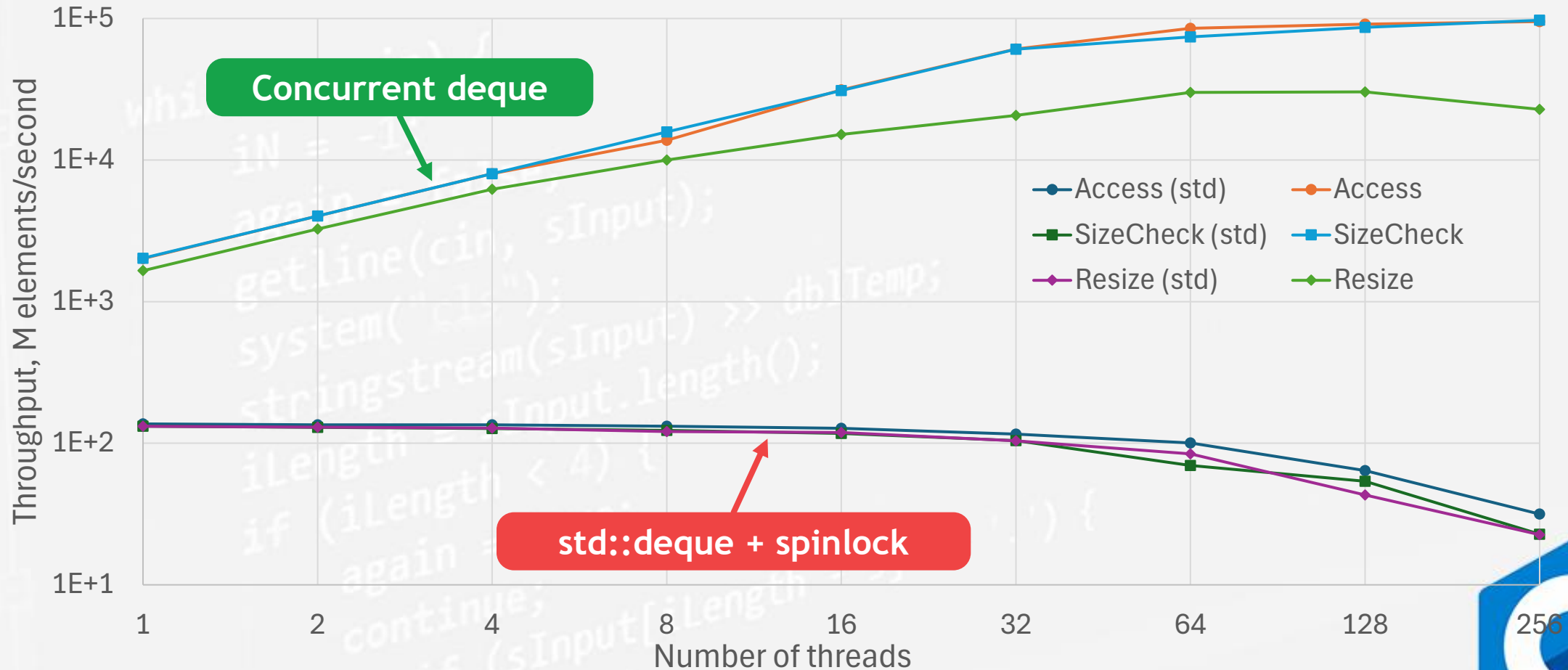


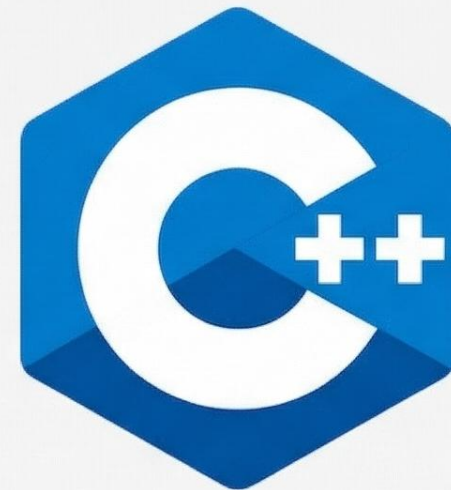
Concurrent Deque - Resize

```
std::lock_guard lock(s_); ← Under lock from now on
if (new_size > cur_size) {
    if (... map is full ...) {
        ... create new_map, old map → garbage list ...
        map_.store(new_map, release); ← Publish
    }
    ... create new blocks ...
    size_.store(new_size, release); ← Publish
} ← unlock is release!
```



Benchmarks vs spinlocked std::deque





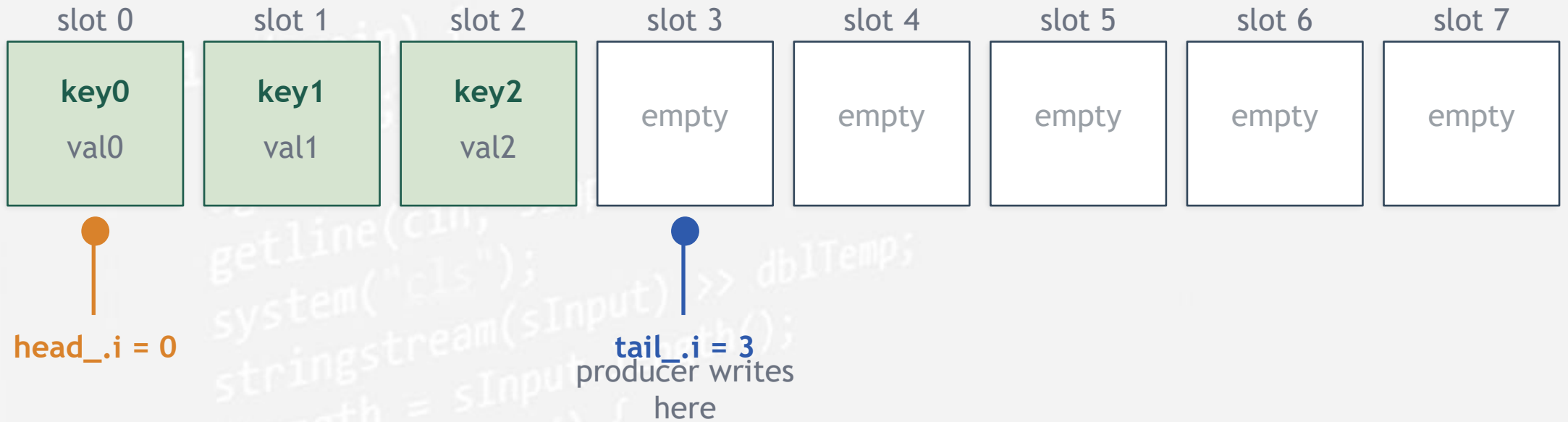
Concurrent Data Structures for Modern Hardware

Concurrent MPMC Queue

- Not-quite-lock-free MPMC queue:
 - spinlock → atomic handoff
- Power-of-2 ring buffer with head/tail indices
- Separate spinlocks for producers (tail) and consumers (head)
 - Spinlock held only during slot claim — released before expensive work
- Per-slot atomics (key + busy flag) for lock-free commit
- Not lock-free - preemption is possible
 - Latency tail needs to be measured
- Requires “reserved” value for keys (e.g., nullptr for T*)



Ring Buffer (Standard Practice)



Each slot contains:

- atomic<Key> key** Non-zero = occupied, zero = empty
- atomic<int> busy (if Value)** 1 = mid-commit, 0 = stable
- Value value (optional)** Constructed/destroyed in place

Occupied
Empty



Two-domain synchronization

Producers (push) — spinlock domain

High contention: every push takes this lock

tail_l
spinlock

tail_i
index, written under lock

Consumers (pop) — spinlock domain

High contention: every pop takes this lock

head_l
spinlock

head_i
index, written under lock

l.release() — spinlock released here; everything below runs with no lock held

Per-slot atomic domain

Low contention: each slot has its own key / busy atomics

Construct value
placement new

key.store(k, release)
→ *busy.store(0)*

Move value out
destruct in place

key.store(0, release)
→ *busy.store(0)*

⚠ Spinlock domains live on separate cache lines — push and pop never contend on the same lock



Push operation: the lock handoff

1 Acquire tail lock

`tail_.l.lock()`

2 Check `key==0 && busy==0`

Verify slot is empty and not mid-commit (with retries)

3 Set `busy=1, ++tail_.i`

Claim slot, advance index

4 Release spinlock

`l.release()` — other pushers unblocked

5 Construct value

placement new — can be expensive, no lock

6 `key.store(k, release)`

Release fence: value visible before key

7 `busy.store(0, release)`

Slot fully committed, consumers can pop

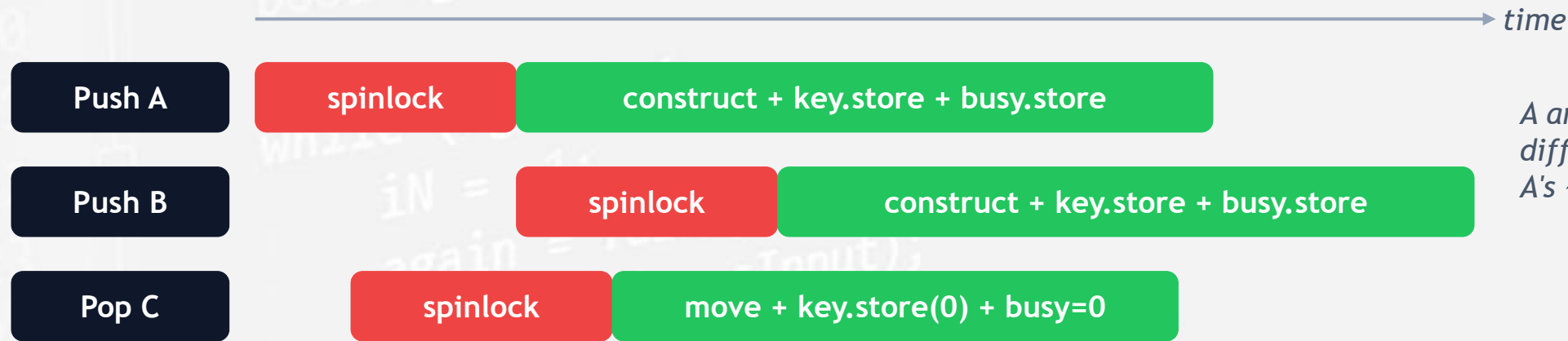
Spinlock domain
lock held: steps 1-4
~4 cheap operations

Atomic domain
no lock: steps 5-7
expensive work

⚠ Lock handoff: release the spinlock before constructing — other pushers proceed while this value is still being built



Contention profile



A and B work in parallel on different slots – B waits only for A's ~4 locked ops

$\text{head_l} \neq \text{tail_l} \rightarrow \text{push and pop never contend on a lock}$

Spinlock domain (high contention)

~3-4 operations: load key, load busy, store busy, increment index

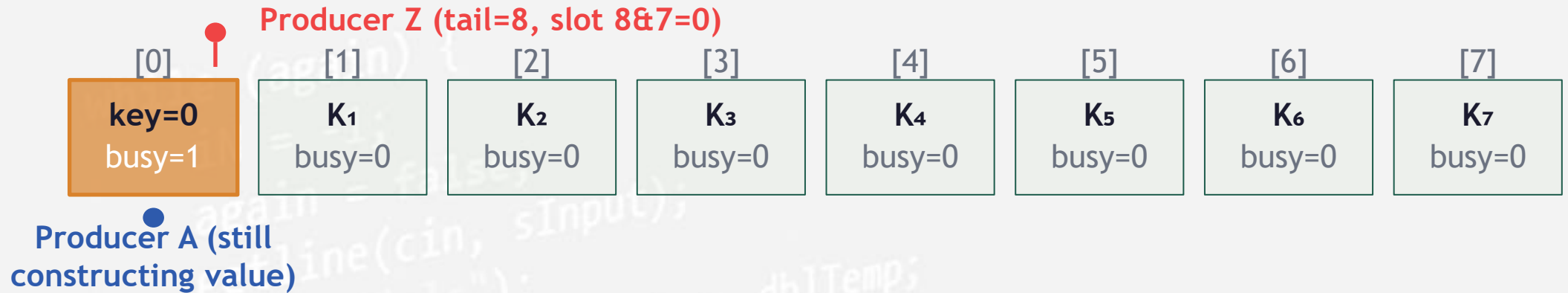
Atomic domain (low contention)

Value construction, key store, busy clear – all on separate slots, rarely contend

⚠ Communicating across lock domains: push key.store(release) $\rightarrow$ pop key.load(acquire)



Producer wraparound race



- 1 Producer A claims slot 0: busy=1, tail++ → releases lock
- 2 Producers B-H push to slots 1-7, all commit successfully
- 3 Producer Z acquires lock. tail_.i=8, slot = 8 & 0x7 = 0
- 4 Z sees key==0 (A hasn't stored key yet!) — looks empty!
- ✓ But busy==1 — Z knows a commit is in flight, retries via NTRY

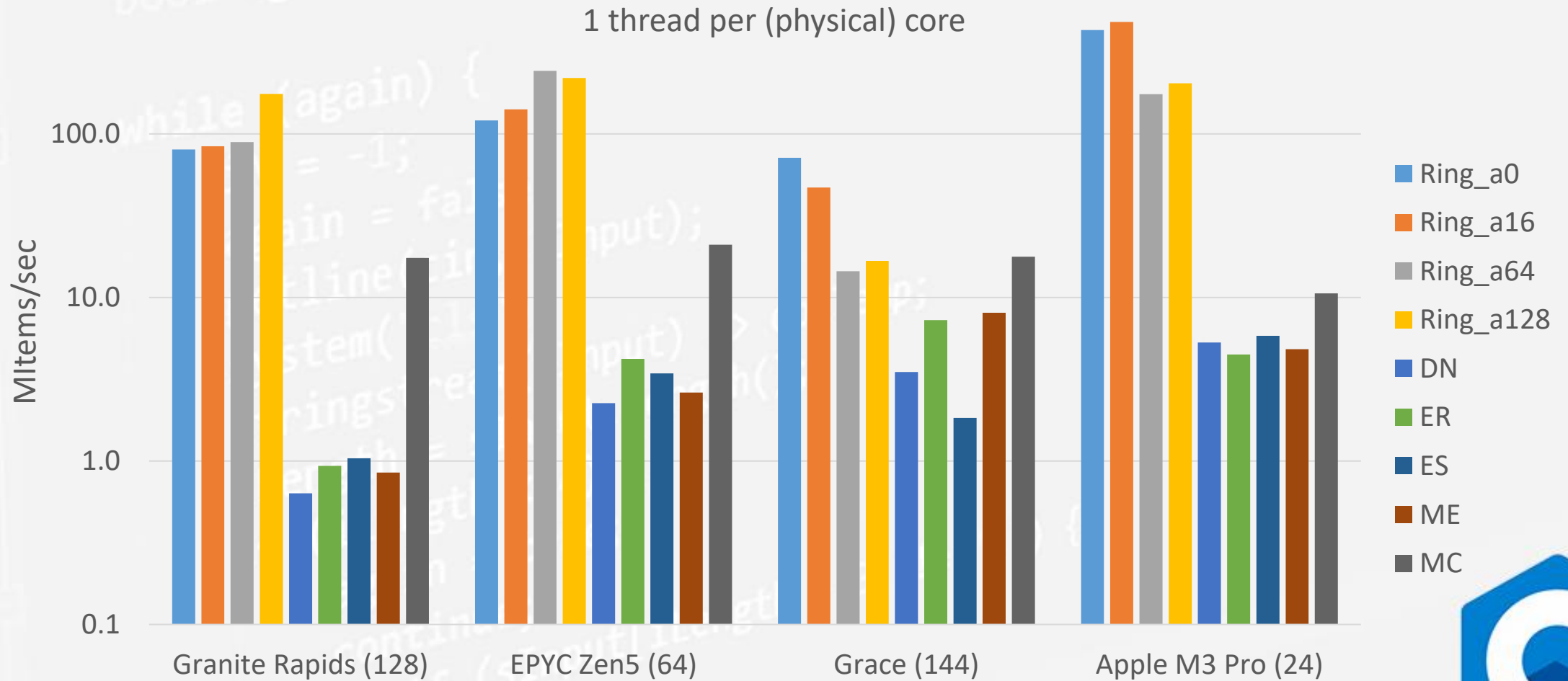


Performance Must Be Measured

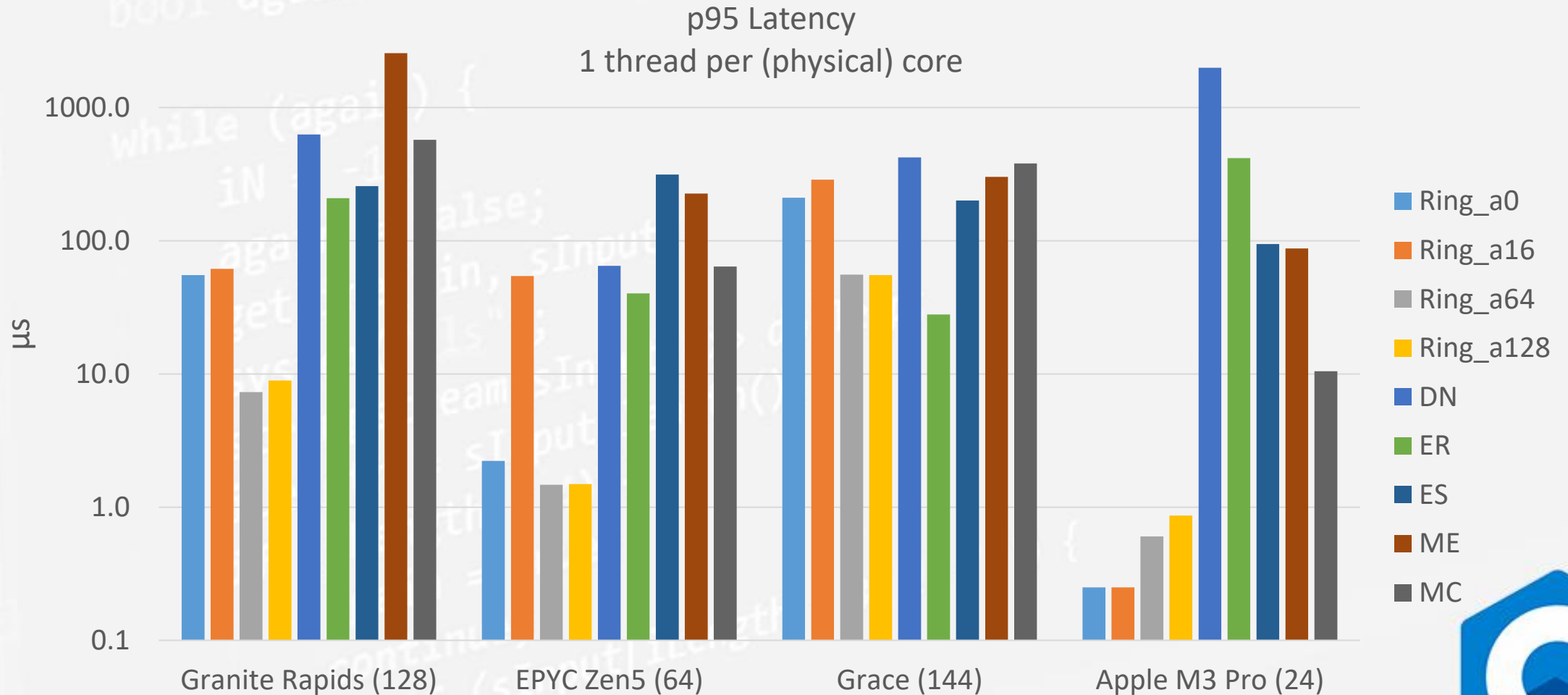
- Ring queue (multiple slot alignments)
- <https://github.com/DNedic/lockfree>
 - Djordje Nedić
- <https://github.com/rigtorp/MPMCQueue>
 - Erik Rigtorp
- https://github.com/erez-strauss/lockfree_mpmc_queue
 - Erez Strauss
- https://github.com/max0x7ba/atomic_queue
 - Maxim Egorushkin
- <https://moodycamel.com/blog/2014/a-fast-general-purpose-lock-free-queue-for-c++.htm>
 - Cameron Desrochers



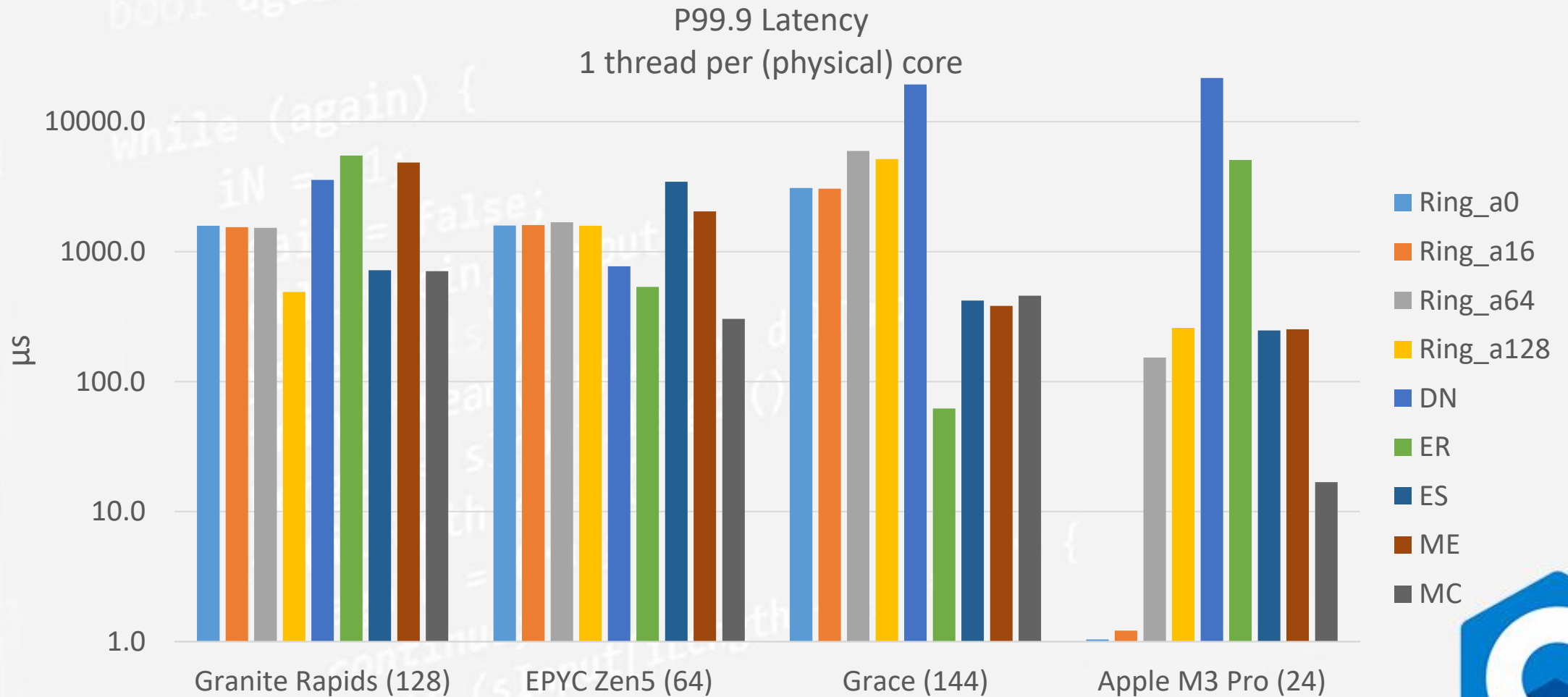
Throughput (loaded queue)



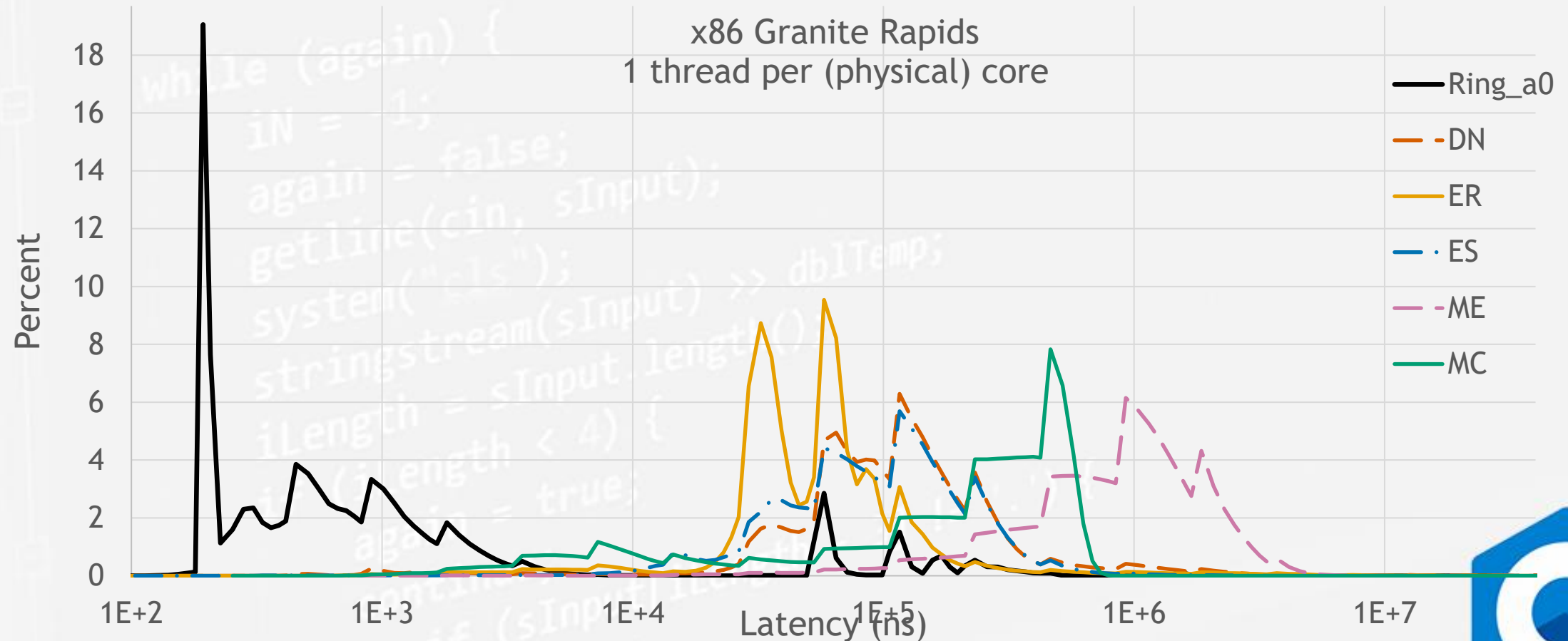
Latency (near-empty queue)



Latency (near-empty queue)



Latency Distribution



Latency tail summary

- Ring queue - lock-based, subject to preemption
- ES queue - fully lock-free, bounded delays

Percentile	R0 (ring, lock-based)	ES (lock-free)
p50	0.51 μ s	77.8 μ s
p90	54.7 μ s	185 μ s
p95	117 μ s	228 μ s
p99	300 μ s	324 μ s
p99.9	1.57 ms	0.51 ms
max	46 ms	9.3 ms

R0 is 3 $\times$ better

ES is 3 $\times$ better

ES is >5 $\times$ better

Grows with time

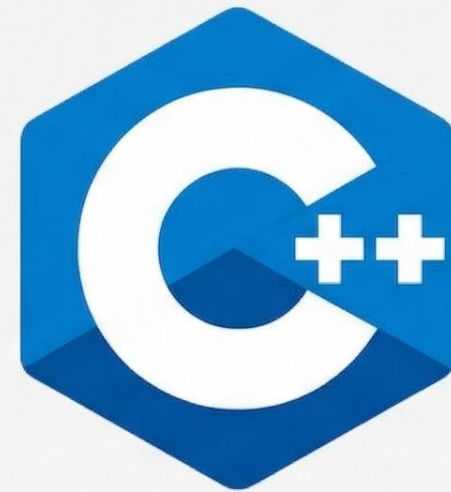
Bounded



Benchmark conclusions

- Design based on the new model appears to work...
 - Spinlocks for high contention
 - Lock-free for low contention
 - Combine locks and atomics for minimum locking
- Preemption is real - lock-based queue has worse outliers
 - At 99.9% latency and beyond - most latencies are better
- Conclusions hold for wide range of hardware
 - x86 and ARM, servers and desktops
- Optimal tuning differs on different hardware:
 - alignment, padding, retries





Lessons Learned

Lock-Free Programming is Dead

- The traditional domain of lock-free programming - shared data under highest contention - is no longer its strength
- High-contention operations are dominated by cache coherency protocols - especially communication latency
- Well-tuned locks with aggressive backoff provide much higher throughput than lock-free RMW operations
 - Other issues with locks remain, in particular, preemption
- “Lock-free style” minimization of critical sections is essential
 - Usually requires atomic variables but atomic $\neq$ lock-free



Long Live Lock-Free Programming

- Avoid using locks for low contention exclusive access
 - Lock cost is irrelevant but locks “poison” adjacent code
 - Locks are “implicit” synchronization - barriers on one shared variable are used to arrange exclusive access to another
 - Locks have branches; branch predictor works but retirement order is program order and lock latencies last longer than most speculations
- Lock-free implementation fuses data and control
 - CPUs can analyze the instruction stream and optimize execution
- Handoff to lock-free synchronization can reduce critical sections



Main Points

- Locks work better at high contention
 - backoff is essential for high-contention writes
 - Read with retries is critical - need to wait for ACKs on RFO
 - Hidden cost of the lock is its release
- Locks can work better with atomics
 - DCLP and other “lock-free style” techniques minimize critical section
- Lock-free works better at low contention
 - Cost of the lock doesn't matter - disruption of execution flow matters
 - Implicit synchronization: lock variable and guarded variable
 - Complex order restrictions disrupt CPU pipeline

<https://github.com/fpikus/ConcurrentCpp>





- 1 Cache contention and pipelines!**
Shared cache lines, RFOs, and pipeline stalls are the real cost
- 2 Spinlocks for high contention**
Slow - but faster than the alternative
- 3 Lock-free for (simple) low contention**
Atomics win when the race is rare and the update is a single CAS
- 4 Atomic handoff across domains (DCLP)**
Peek the atomic outside the lock; take the lock only when it pays
- 5 Locks + atomics = less locking**
Hold the lock briefly, publish with release, read with acquire

<https://github.com/fpikus/ConcurrentCpp>